SIEMENS

TIA Portal Openness object 1

General functions 2

Functions for projects and project data 3

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 TIA Portal Openness object 7
1.1 TIA Portal Openness object model 7
1.2 Blocks and types of the TIA Portal Openness object model 11
1.3 Hierarchy of hardware objects of the object model 16
1.4 Object list 18

2 General functions 25
2.1 Introduction 25
2.2 Information about installed TIA Portal Openness versions 25
2.3 TIA Portal Openness IntelliSense support 26
2.4 Standard libraries 27
2.5 Use of the code examples 28
2.6 Example program 31
2.7 Programming steps 38
2.8 Connecting to the TIA Portal 39
2.9 TIA Portal Openness firewall 43
2.10 Event handlers 45
2.11 Program-controlled acknowledgement of dialogs with system events 48
2.12 Terminating the connection to the TIA Portal 51
2.13 Diagnostic interfaces on TIA Portal 52
2.14 Exclusive access 59
2.15 Dynamic behaviours 61
2.16 Working with associations 66
2.17 Working with compositions 67
2.18 Verifying object equality 68
2.19 Read operations for attributes 69
2.20 Transaction handling 71
2.21 Creating a DirectoryInfo/FileInfo object 77
2.22 Self-description support for attributes, navigators, actions, and services 80
2.23 Setting attributes of Blocks, DBs & UDTs 86
2.24 Notes on performance of TIA Portal Openness 88

3 Functions for projects and project data 89
3.1 Opening a project 89
3.2 Creating a project 95
3.3 Accessing general settings of the TIA Portal 98
3.4 Archiving and Retrieving a project 101
3.5 Accessing read-only TIA Portal project 107
3.6 Accessing languages 108
3.7 Determining the object structure and attributes 111

3.8 Access software target 113
3.9 Accessing and enumerating multilingual texts 115
3.10 Updating project properties 116
3.11 Read project related attributes 118
3.12 Deleting project graphics 122
3.13 Compiling a project 123
3.14 Modifying project in UMAC 126
3.15 Saving a project 127
3.16 Closing a project 129
3.17 Export/Import system diagnostics settings 129
3.18 Identifying cross session object for Hardware & Software 131
3.19 Managing exception for missing component in project 135
3.20 Accessing multilingual support 136

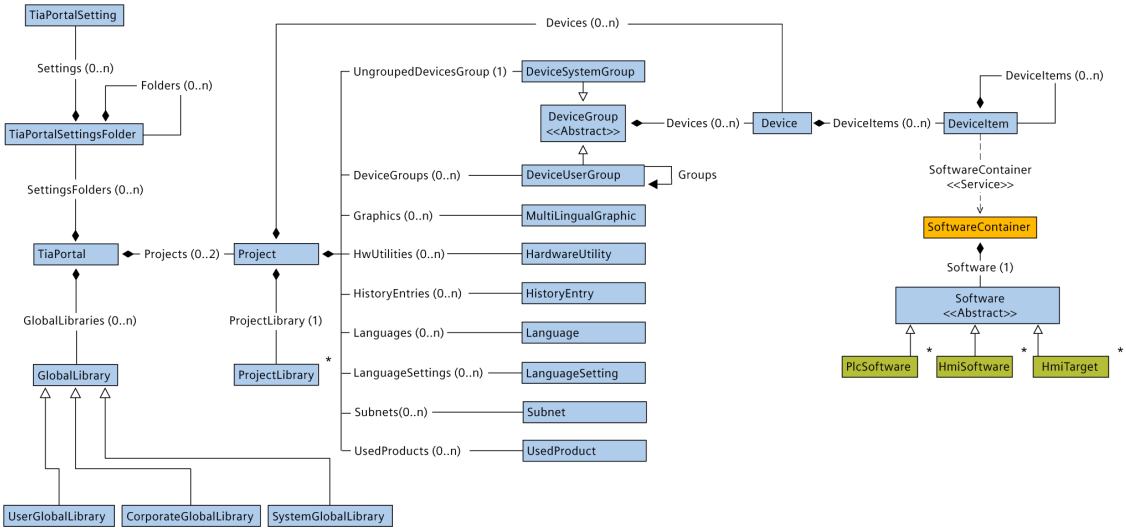
TIA Portal Openness object

1

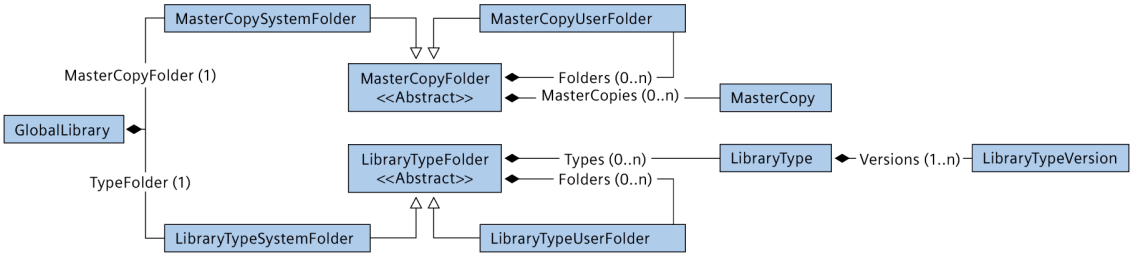
1.1 TIA Portal Openness object model

Overview

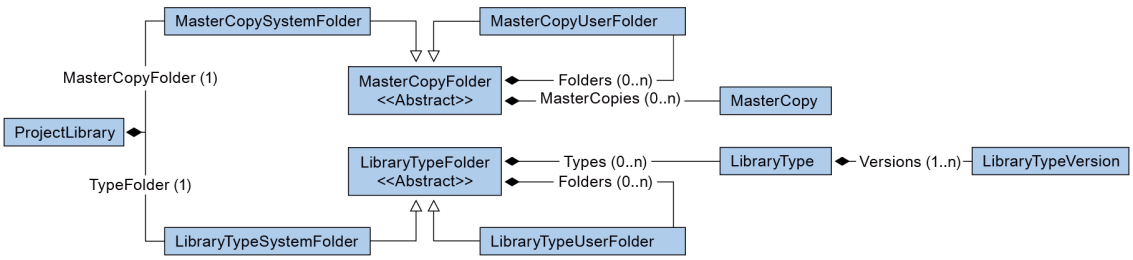
The following diagram describes the highest level of the TIA Portal Openness object model:



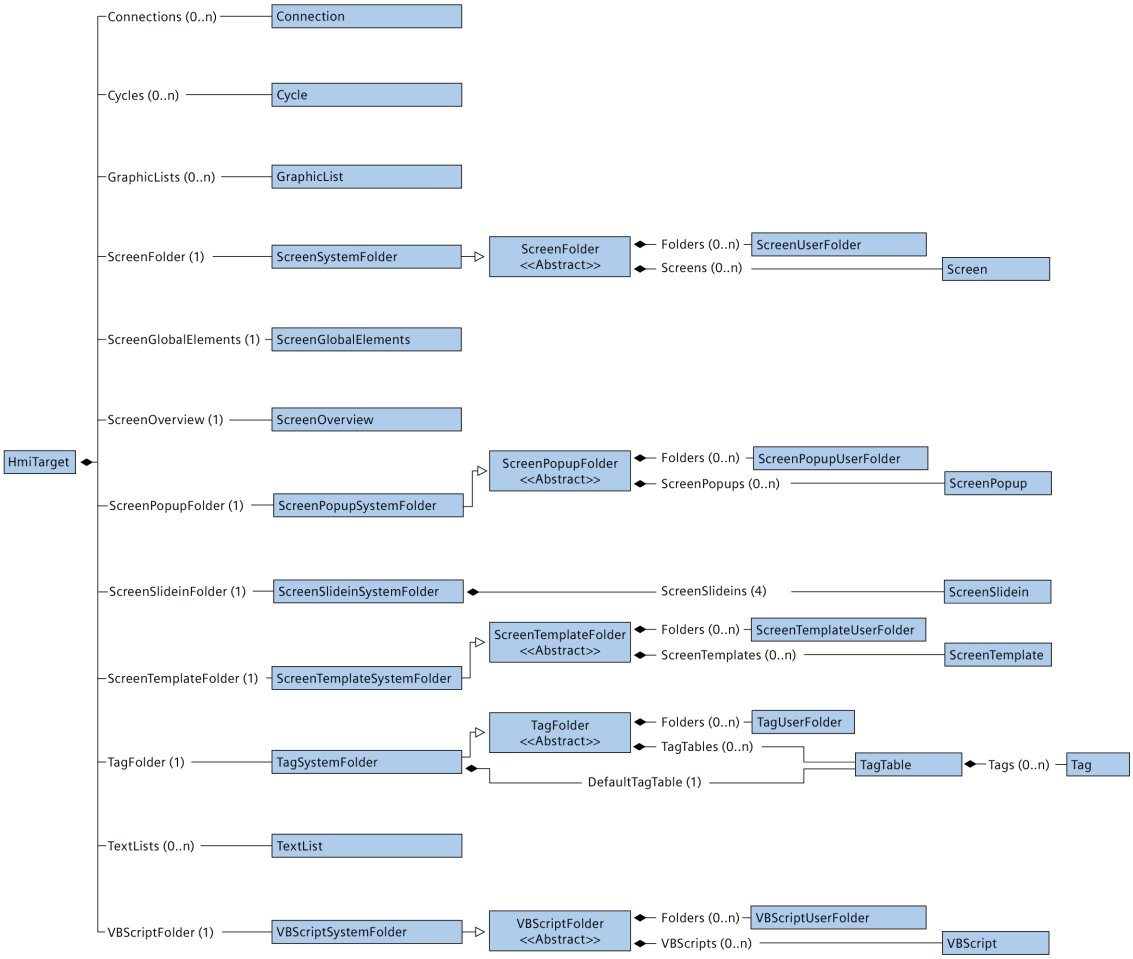
The following diagram describes the objects which are located under GlobalLibrary.



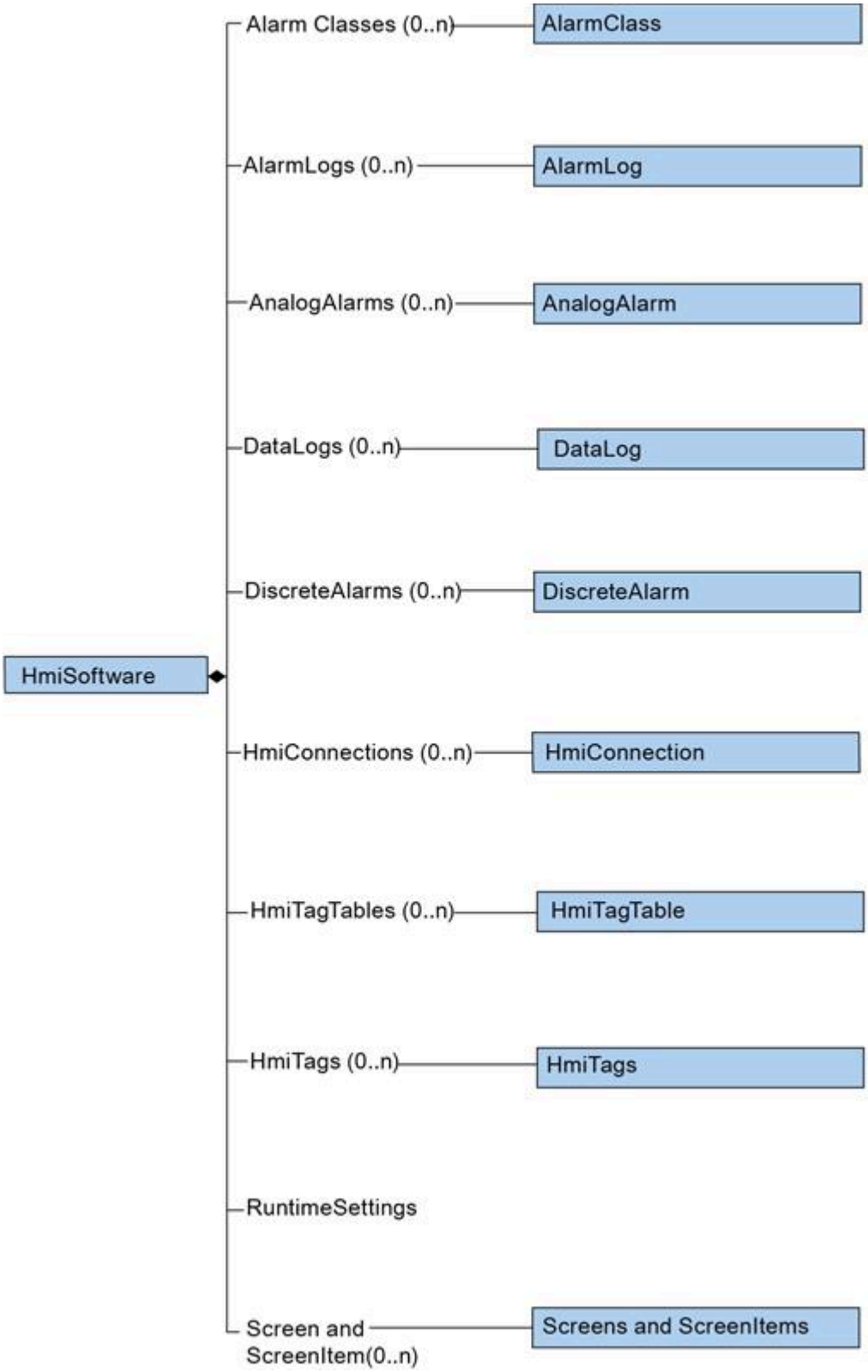
The following diagram describes the objects which are located under ProjectLibrary.



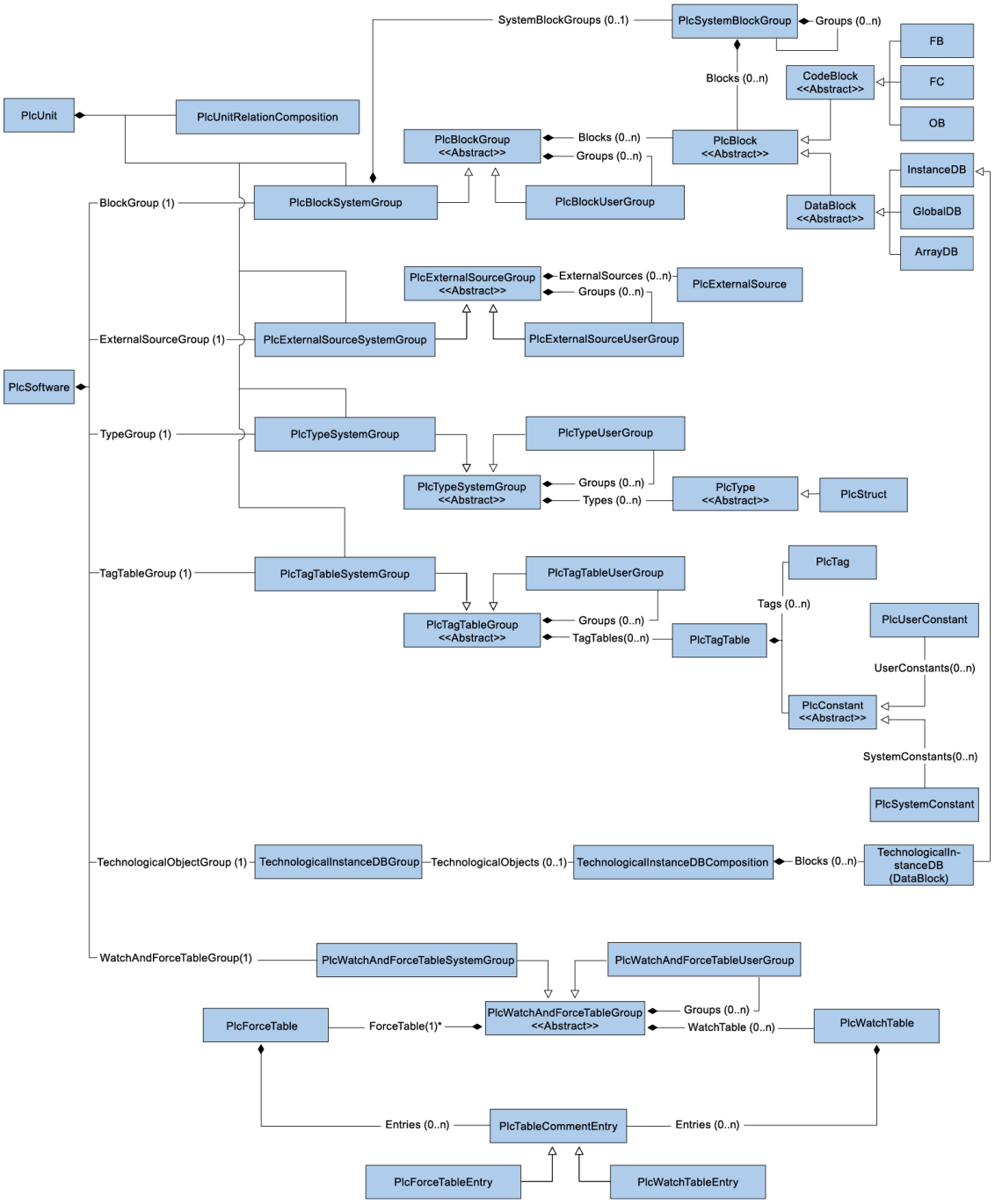
The following diagram describes the objects which are located under HmiTarget.



The following diagram describes the object which are located under HmiSoftware.



The following diagram describes the objects which are located under PlcSoftware.



Note

*The Force Table must be in PlcWatch and ForceTableSystemGroup

Access to objects in lists

You have the following options for addressing an object in a list:

- Address via the index. The counting within the lists starts with `0`.

- Use `Find` method.
  Use this method to address an object via its name. You can use this method for an composition or list. The `Find` method is not recursive.

  Example:

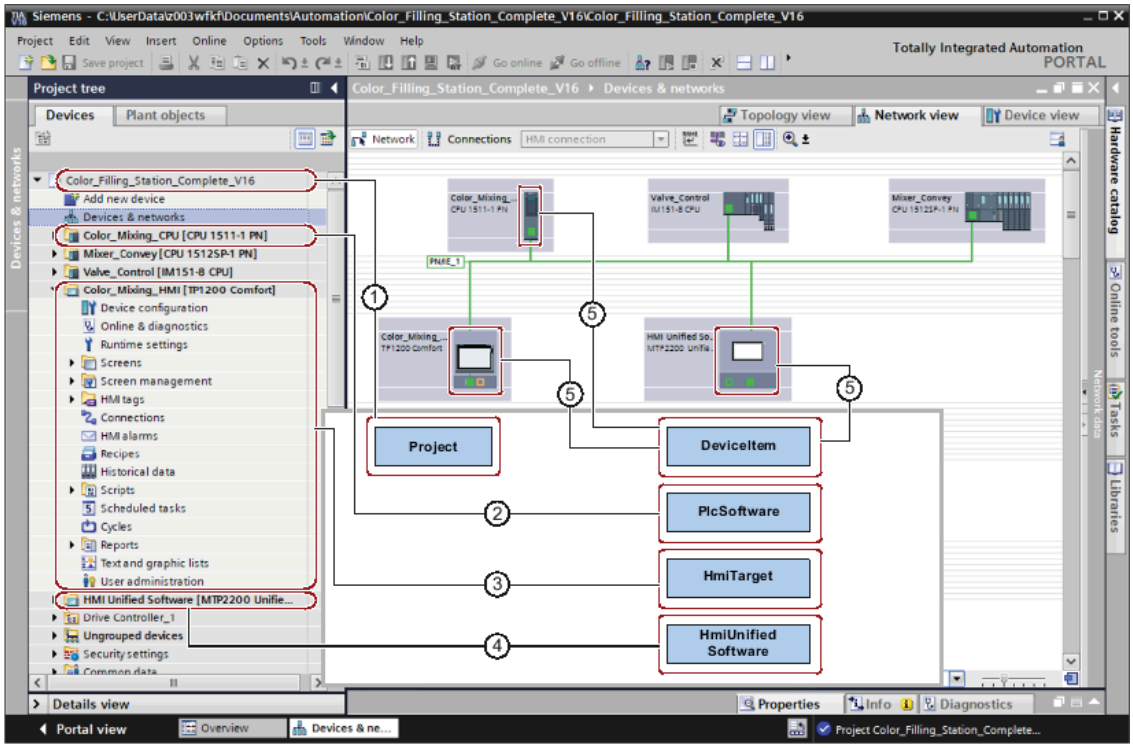
  ```
  ScreenComposition screens = folder.Screens;

  Screen screen = screens.Find("myScreen");
  ```

- Use symbolic names.

Relationship between TIA Portal and TIA Portal Openness object model

The figure below shows the relationship between the object model and a project in the TIA Portal:



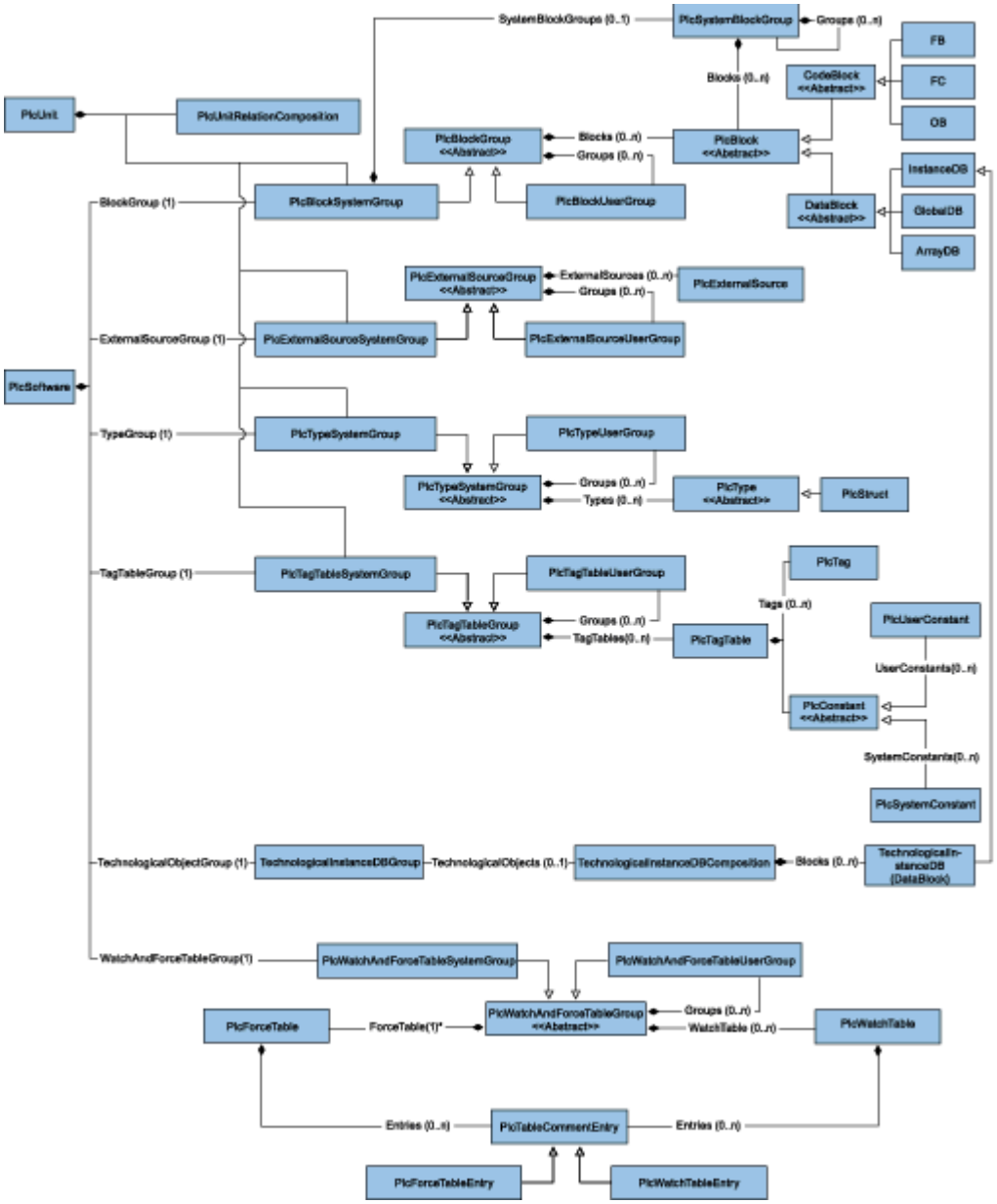
See also

Blocks and types of the TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.2 Blocks and types of the TIA Portal Openness object model

Introduction

The following diagram describes the domain model of the PLCs to give an overview of the current modeling in TIA Portal Openness.



Note

* The Force Table must be in PlcWatch and ForceTableSystemGroup

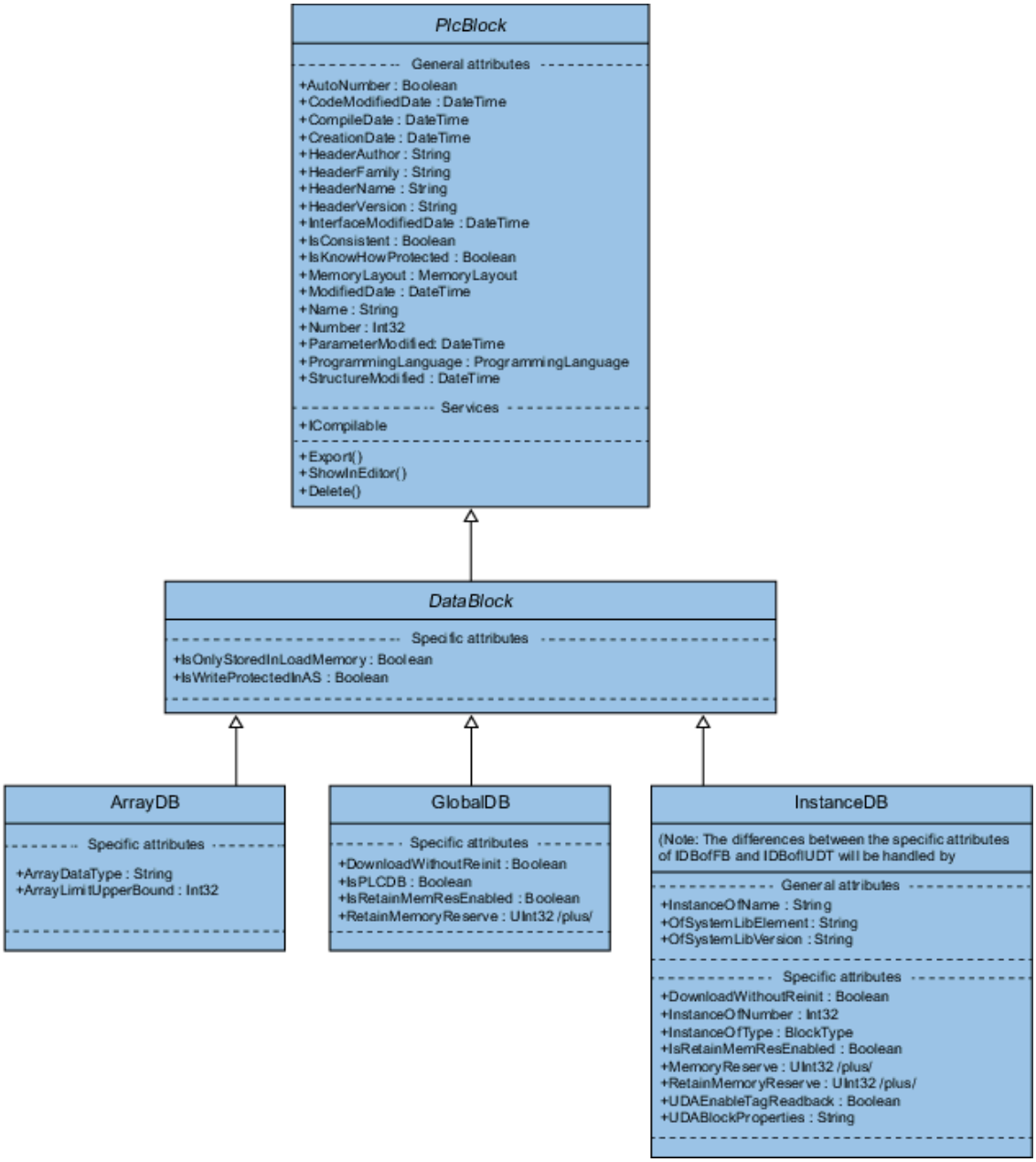
Representation of blocks and types in the TIA Portal Openness API

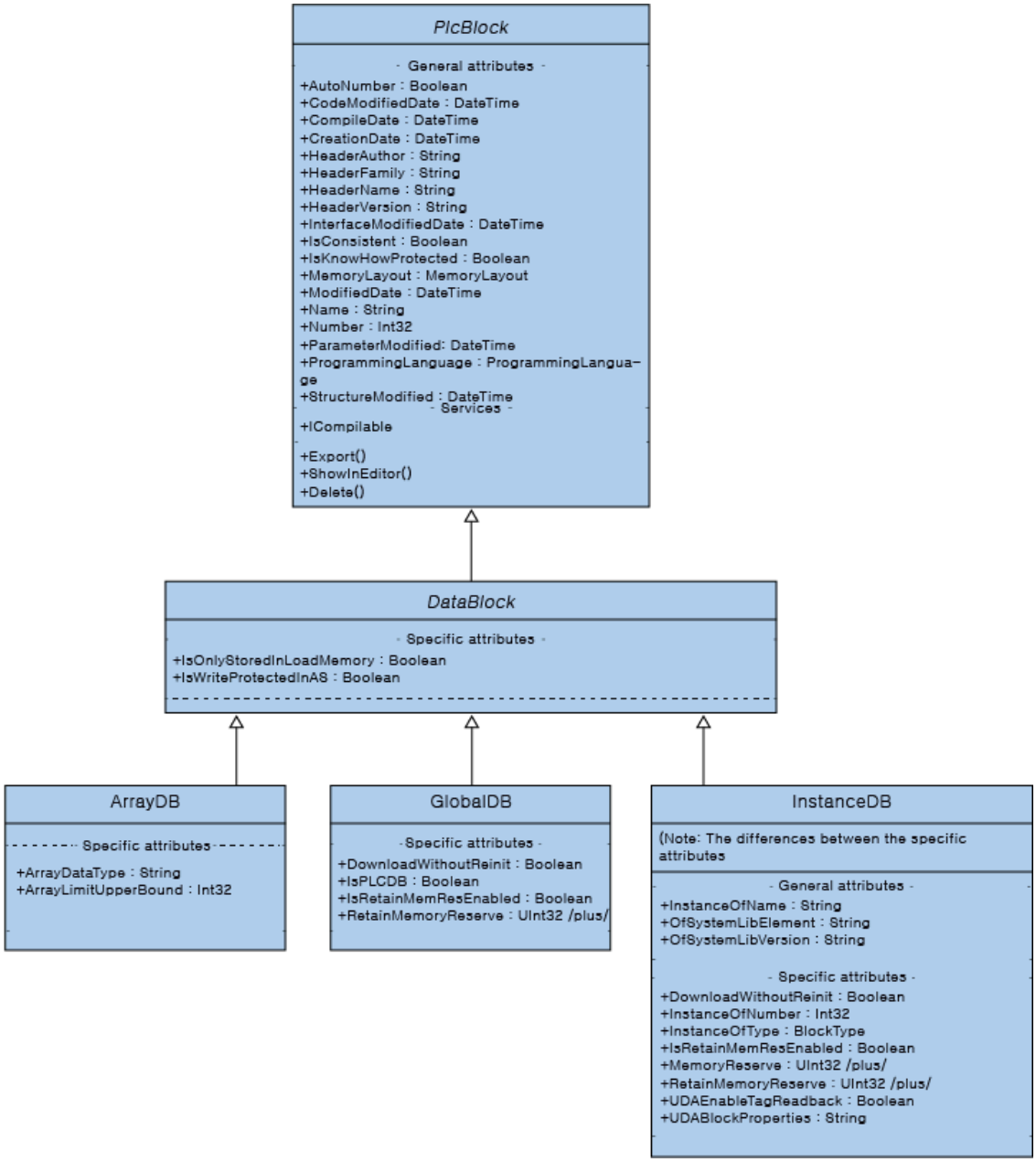
The simplified model part of blocks and for the structure is based on the attributes in the TIA Portal Openness API. These classes provide the export function and for blocks also the compile function.

Class diagrams

In the TIA Portal Openness object model all classes are defined as abstract which aren't directly instantiated.

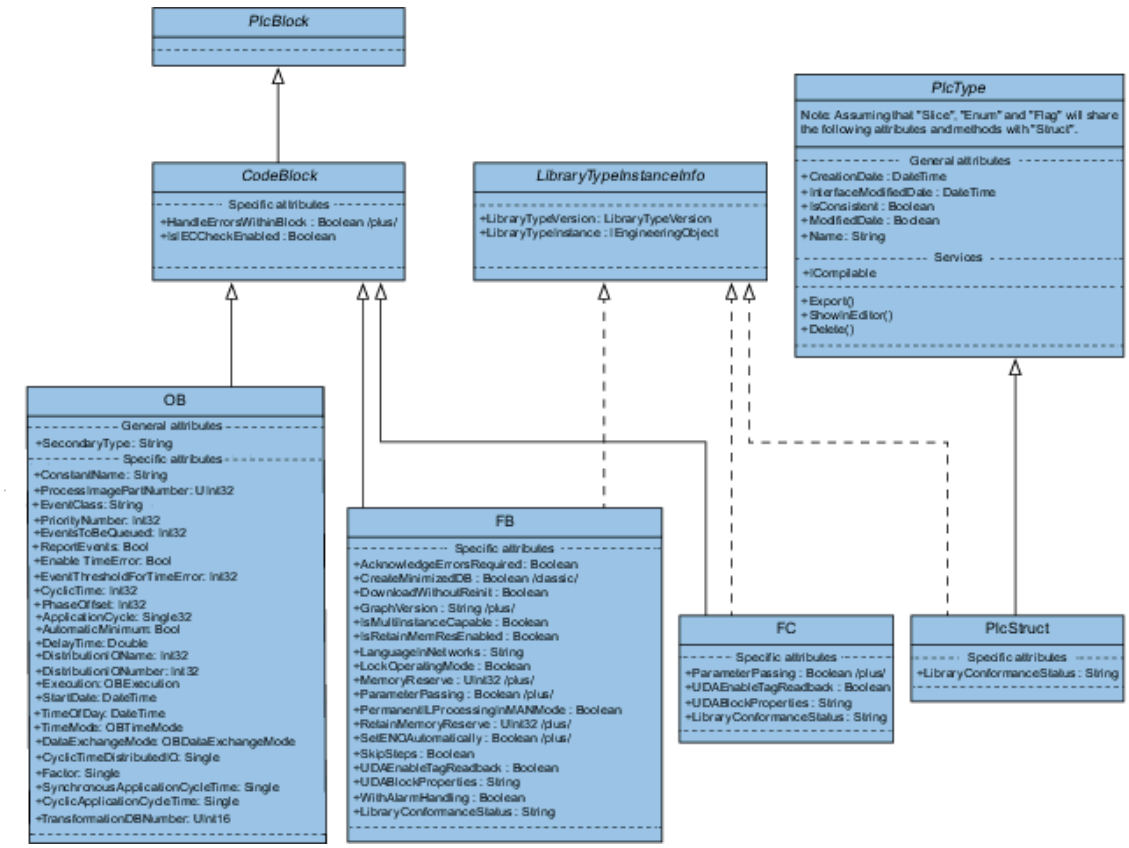
Data


PlcBlock
General attributes
+AutoNumber : Boolean
+CodeModifiedDate : DateTime
+CompileDate : DateTime
+CreationDate : DateTime
+HeaderAuthor : String
+HeaderFamily : String
+HeaderName : String
+HeaderVersion : String
+InterfaceModifiedDate : DateTime
+IsConsistent : Boolean
+IsKnowHowProtected : Boolean
+MemoryLayout : MemoryLayout
+ModifiedDate : DateTime
+Name : String
+Number : Int32
+ParameterModified: DateTime
+ProgrammingLanguage : ProgrammingLanguage
+StructureModified : DateTime
Services
+ICompilable
+Export()
+ShowInEditor()
+Delete()
DataBlock
Specific attributes
+IsOnlyStoredInLoadMemory : Boolean
+IsWriteProtectedInAS : Boolean
ArrayDB
Specific attributes
+ArrayDataType : String
+ArrayLimitUpperBound : Int32
GlobalDB
Specific attributes
+DownloadWithoutReinit : Boolean
+IsPLCDB : Boolean
+IsRetainMemResEnabled : Boolean
+RetainMemoryReserve : UInt32 /plus/
InstanceDB
(Note: The differences between the specific attributes of IDBofFB and IDBofUDT will be handled by
General attributes
+InstanceOfName : String
+OfSystemLibElement : String
+OfSystemLibVersion : String
Specific attributes
+DownloadWithoutReinit : Boolean
+InstanceOfNumber : Int32
+InstanceOfType : BlockType
+IsRetainMemResEnabled : Boolean
+MemoryReserve : UInt32 /plus/
+RetainMemoryReserve : UInt32 /plus/
+UDAEnableTagReadback : Boolean
+UDABlockProperties : String



Representation of block diagram

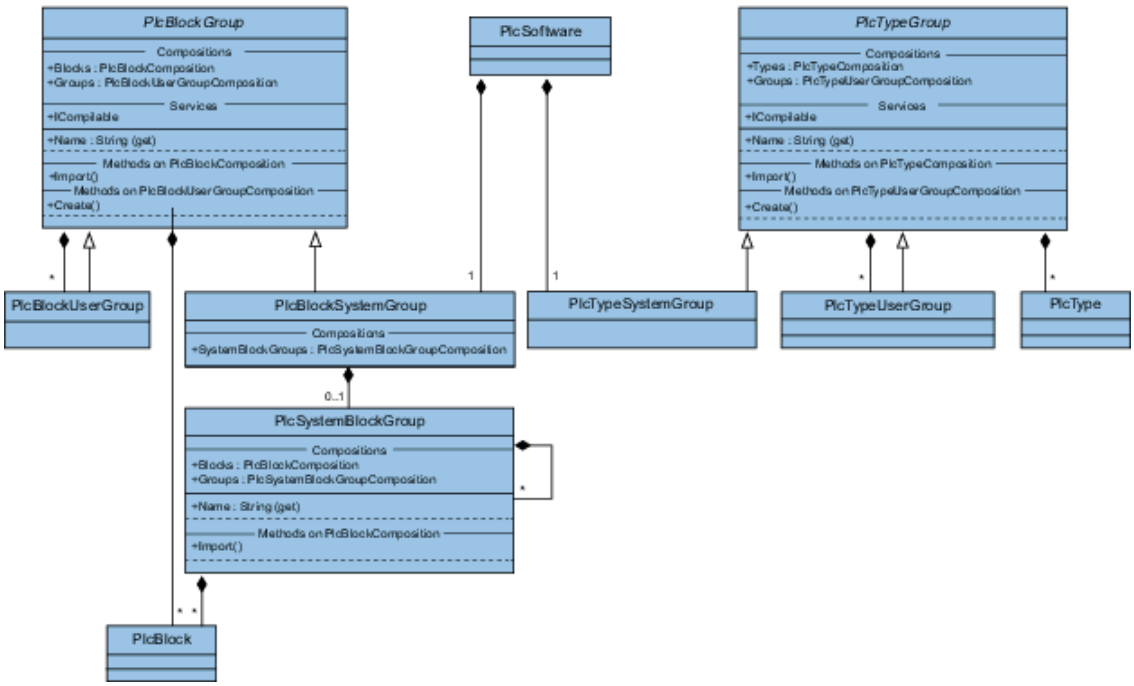
Code and Type



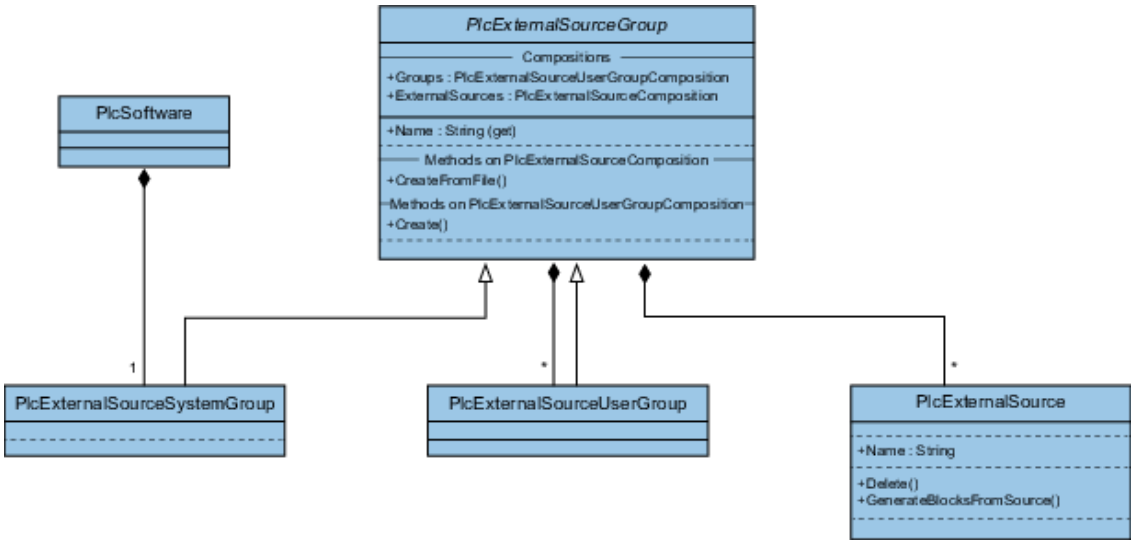
Representation of groups for blocks and types in the TIA Portal Openness API

The two high level groups "PlcBlocks" ("Program blocks" in the GUI of TIA Portal) and "PlcTypes" ("Plc data types" in the GUI of TIA Portal) contain blocks and type definitions. These groups provide the import and the compile functions for blocks. Due to the fact that most of the methods of functionalities of the groups are only achievable via collections, there is an "embedded" or "compacted" representation of the collections and their methods at the "host" classes.

Blocks and Types



External Sources



See also

TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.3 Hierarchy of hardware objects of the object model

Relation between the visible elements in the TIA Portal and the modeled elements in the object model

Hardware object	Explanation
Device (Device)	The container object for a central or distributed configuration.
Device item (DeviceItem)	Each device item object has a container object. The logical relation is "Items".

To understand the hierarchy relationship of hardware objects between what is displayed in the TIA Portal view and what is modeled in the TIA Portal Openness, it is necessary to consider some aspects of the hierarchy of hardware objects.

Example: A HardwareObject has the items link. The Device extends the HardwareObject and has the UnpluggedItems link. The Device is the container object for a station.

The DeviceItem extends the HardwareObject and has the Container link. Every plugged DeviceItem object has got a container object which is a HardwareObject. The reverse relation is Items.

An unplugged DeviceItem object that does not have a container is found via the link UnpluggedItems of the Device.

The container relation is comparable to the relation of the modules for the device item objects.

Example: A device includes one or more racks. A rack includes modules. A module includes submodules.

This is the relation similar to the representation in the network view and device view of the TIA Portal. The "PositionNumber" attribute of a device item is unique in the items area, within a container.

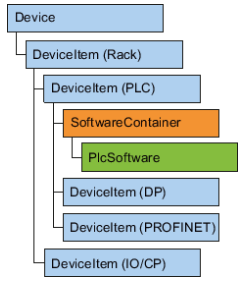
The parent-child relation between device item objects is a purely logical relation in the object model. A child cannot exist without its parents.

- If a submodule is modeled as part of a module (child), the submodule cannot be removed without the module.
- A module or submodule that can be added and removed is a child of the device object

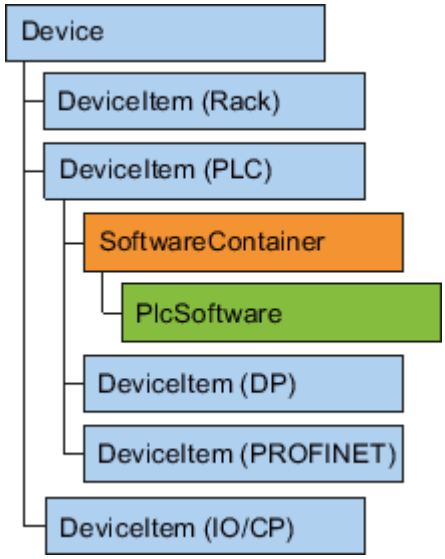
Hierarchy relationships of PLC devices

The diagrams below show the hierarchy relationship between devices, and device items of PLC and HMI devices.

Hierarchy when iterating via .Items	

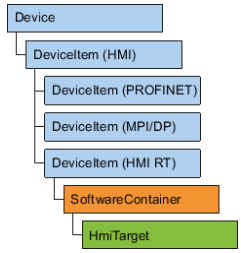


Hierarchy when iterating via .DeviceItems

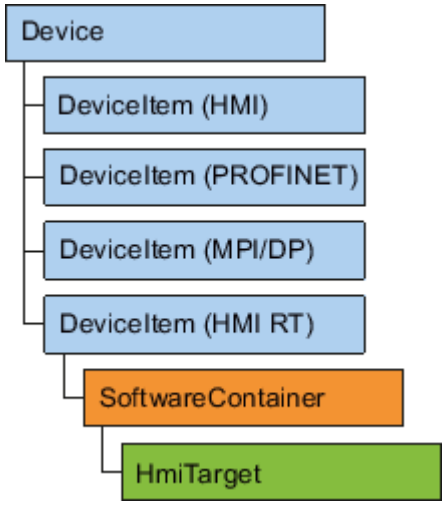


Hierarchy relationships of HMI devices

Hierarchy when iterating via .Items



Hierarchy when iterating via .DeviceItems



See also

TIA Portal Openness object model
Blocks and types of the TIA Portal Openness object model

1.4 Object list

Introduction

The following tables show the available objects up to and including Runtime Advanced, and indicate whether these objects are supported by TIA Portal Openness.

Neither Runtime Professional nor device proxy files are supported by TIA Portal Openness in WinCC.

Objects

You can control the following project data depending on which HMI device you are using:

Screens

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Screen	Yes	Yes	Yes	Yes
Global screen	Yes	Yes	Yes	Yes
Templates	Yes	Yes	Yes	Yes
Permanent area	No	Yes	Yes	Yes
Pop-up screen	No	Yes	Yes	Yes
Slide-in screen	No	Yes	Yes	Yes

Screen objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Line	Yes	Yes	Yes	Yes
Polyline	No	Yes	Yes	Yes
Polygon	No	Yes	Yes	Yes
Ellipse	Yes	Yes	Yes	Yes
Ellipse segment	No	No	No	No
Circle segment	No	No	No	No
Elliptical arc	No	No	No	No
Camera view	No	No	No	No

Circular arc	No	No	No	No
Circle	Yes	Yes	Yes	Yes
PDF view	No	No	No	No
Rectangle	Yes	Yes	Yes	Yes
Connector	No	No	No	No
Text field	Yes	Yes	Yes	Yes
Graphic view	Yes	Yes	Yes	Yes
Pipe	No	No	No	No
Double T-piece	No	No	No	No
T-piece	No	No	No	No
Pipe bends	No	No	No	No
I/O field	Yes	Yes	Yes	Yes
Date/time field	Yes	Yes	Yes	Yes
Graphic I/O field	Yes	Yes	Yes	Yes
Editable text field	No	No	No	No
List box	No	No	No	No
Combo box	No	No	No	No
Button	Yes	Yes	Yes	Yes
Round button	No	No	No	No
Illuminated button	No	No	Yes	No
Switch	Yes	Yes	Yes	Yes
Symbolic I/O field	Yes	Yes	Yes	Yes

Key-operated switch	No	No	Yes	No
Bar	Yes	Yes	Yes	Yes
Symbol library	No	Yes	Yes	Yes
Slider	No	Yes	Yes	Yes
Scroll bar	No	No	No	No
Check box	No	No	No	No
Option buttons	No	No	No	No
Gauge	No	Yes	Yes	Yes
Clock	No	Yes	Yes	Yes
Memory space view	No	No	No	No
Function keys	Yes	Yes	Yes	Yes
Faceplate instances	No	Yes	Yes	Yes
Screen window	No	No	No	No
User view	Yes	Yes	Yes	Yes
HTML Browser	No	No	No	No
Print job/script diagnostics	No	No	No	No
Recipe view	No	No	No	No
Alarm view	No	No	No	No
Alarm indicator	No	No	No	No
Alarm window	No	No	No	No
Criteria analysis view	No	Yes [1)]	Yes	Yes
ProDiag overview	No	Yes [1)]	Yes	Yes

GRAPH overview	No	Yes [1]	Yes	Yes
PLC code view	No	Yes [1]	Yes	Yes
f(x) trend view	No	No	No	No
f(t) trend view	No	No	No	No
Table view	No	No	No	No
Value table	No	No	No	No
Media Player	No	No	No	No
Channel diagnostics	No	No	No	No
WLAN reception	No	No	No	No
Zone name	No	No	No	No
Zone signal	No	No	No	No
Effective range name	No	No	No	No
Effective range name (RFID)	No	No	No	No
Effective range signal	No	No	No	No
Charge condition	No	No	No	No
Handwheel	No	No	Yes	No
Help indicator	No	No	No	No
Sm@rtClient view	No	No	No	No
Status/Force	No	No	No	No
System diagnostic view	No	No	No	No
System diagnostic window	No	No	No	No

1) Only Mobile Panels with the device version greater than 12.0.0.0 support this screen object Dynamic

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Display	Yes	Yes	Yes	Yes
Operability	No	Yes	Yes	Yes
Visibility	Yes	Yes	Yes	Yes
Movements	Yes	Yes	Yes	Yes

Additional objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Groups	Yes	Yes	Yes	Yes
Soft keys	Yes	Yes	Yes	Yes
Cycles	Yes	Yes	Yes	Yes
VB scripts	No	Yes	Yes	Yes
Function lists	Yes	Yes	Yes	Yes
Project graphics	Yes	Yes	Yes	Yes

Tags

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multiplex tags	Yes	Yes	Yes	Yes
Arrays	Yes	Yes	Yes	Yes
User data types	Yes	Yes	Yes	Yes
Internal	No	Yes	Yes	Yes

Points of use of elementary data types	Yes	Yes	Yes	Yes
Points of use of user data types	Yes	Yes	Yes	Yes
Points of use of arrays	Yes	Yes	Yes	Yes

TIA Portal Openness also supports all value ranges which are supported by the communication drivers.

Connections

TIA Portal Openness supports non-integrated connections that are also supported by the respective HMI devices. You can find additional information in the online help for the TIA Portal under "Process visualization > Controller communication > Device-dependent".

Lists

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Text lists	Yes	Yes	Yes	Yes
Graphic lists	Yes	Yes	Yes	Yes

Texts

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multilingual texts	Yes	Yes	Yes	Yes
Formatted texts and their instances	No	Yes	Yes	Yes

General functions 2

2.1 Introduction

Overview

TIA Portal Openness supports a selection of functions for defined tasks that you can call outside the TIA Portal by means of the TIA Portal Openness API.

Note

If a previous version of TIA Portal Openness is already installed, the current version will be installed side by side.

You are provided with an overview of the typical programming steps in the sections below. You can learn how the individual code sections interact and how to integrate the respective functions into a complete program. You also get an overview of the code components that have to be adapted for each task.

Example program

The individual programming steps are explained using the "Creating API access in a console application" function as an example. You integrate the provided functions in this program code and adapt the respective code components for this task.

Functions

The section below lists the functions for defined tasks that you can call with TIA Portal Openness outside the TIA Portal.

See also

Applications
Object list

2.2 Information about installed TIA Portal Openness versions

Each installed TIA Portal version provides information about available, compatible TIA Portal Openness versions in the Windows registry. This enables TIA Portal Openness applications to automatically start or connect to matching TIA Portal instances.

The relevant TIA Portal Openness registry keys are in the following path in the Windows registry:

HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness

The registry key "Openness" contains the following subkeys and values:

- Version-independent subkey "AllowList" for the TIA Portal Openness firewall (only from TIA Portal V21 onwards)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList
  - See also TIA Portal Openness firewall
- Subkeys for each installed TIA Portal version: "xx.xx" (technical version number)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx
  - The string value "PortalVersion" for a display string of that TIA Portal version (only from TIA Portal V21 onwards)
- Subkey "PublicAPI"
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI
- Subkeys for each provided TIA Portal Openness version in that specific TIA Portal version: "xx.xx.xx.xx" (technical version number)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx
  - The string value "EngineeringVersion" for a display string of that TIA Portal Openness version
- Subkeys for each supported .NET target framework version in that specific TIA Portal Openness version using the Microsoft Target Framework Moniker (TFM) which is a standardized token format for specifying the target framework of a .NET library (only from TIA Portal V21 onwards): "netxxx"
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx\netxx
  - The string value "AssemblyVersion" which the Microsoft CLR uses to load the exact version
  - The string value "PublicKeyToken" as part of the strong-named assembly to create a unique identity to prevent assembly conflicts
  - The string value "Siemens.Engineering.Base" pointing to the custom installation path

Note

Regardless which TIA Portal products are installed, the Windows registry only provides the installation path to the base assembly "Siemens.Engineering.Base".

See also

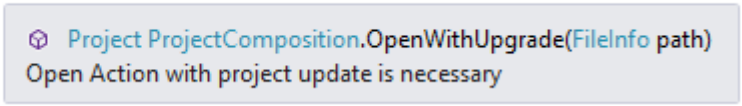
Connecting to the TIA Portal

2.3 TIA Portal Openness IntelliSense support

Application

The Intellisense support of TIA Portal Openness helps you at available attributes or methods via tooltip information. It could contain information about the number, names and types of the required parameters. At the following example the bold parameter in the first line indicates the next parameter that is required as you type the function.

```
project = tiaPortal.Projects.OpenWithUpgrade(projectInfo);
```



You can manually invoke Parameter Info by clicking Edit IntelliSense/Parameter Info, typing CTRL+SHIFT+SPACE, or clicking the Parameter Info button on the editor toolbar.

2.4 Standard libraries

Insert the following namespace statements at the beginning of the relevant code example to ensure that the code examples work:

```
Copies the following program code to clipboard.
using System;
using System.Collections.Generic;
using System.IO;
using Siemens.Engineering;
using Siemens.Engineering.Cax;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Compare;
using Siemens.Engineering.Download;
using Siemens.Engineering.Hmi;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.RuntimeScripting;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Extensions;
using Siemens.Engineering.HW.Features;
```

```
using Siemens.Engineering.HW.Utilities;
using Siemens.Engineering.Library;
using Siemens.Engineering.Library.MasterCopies;
using Siemens.Engineering.Library.Types;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.TechnologicalObjects;
using Siemens.Engineering.SW.TechnologicalObjects.Motion;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Upload;
```

2.5 Use of the code examples

Structure of the code-snippets

Each code-snippet in this documentation is implemented as a function without return value with an object reference as transfer parameter. Disposing of objects is omitted for the sake of readability. Objects of the TIA Portal are addressed by their name using the `Find` method.



```
Copies the following program code to clipboard.
//Deletes a single screen from a user folder or a system folder
private static void DeleteScreenFromFolder(HmiTarget hmiTarget)
{
    //The screen "MyScreen" will be deleted if it is existing in the folder "myScreenFolder".
    //If "myScreen" is stored in a subfolder of "myScreenFolder" it will not be deleted.
    string screenName = "MyScreen";
    ScreenUserFolder folder = hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
```

```
    if (screen != null)
    {
        screen.Delete();
    }
}
```

You need the following to execute this code-snippet:

- A WinCC project with an HMI device that includes a group with at least one screen.
- A function that instantiates the HMI device.

Note

When you specify directory paths, use the absolute directory path, for example, "C:/path/file.txt".

Relative directory paths are only allowed in the XML files for import and export, for example, "file.txt" or "C:/path01/.../path02/file.txt".

Example for execution of the code-snippet

Use the following example to execute the code-snippet "DeleteScreenFromFolder" as part of the "Hello TIA" example program:

```
Copies the following program code to clipboard.
//In the sample program "Hello TIA" replace the function call
//"IterateThroughDevices(project)" by the following functions
calls:
    HmiTarget hmiTarget = GetTheFirstHmiTarget(project);
    DeleteScreenFromFolder(hmiTarget);
//Put the following function definitions before or after the
//function definition of "private static void
IterateThroughDevices(Project project)":
private static HmiTarget GetTheFirstHmiTarget(Project
project)
{
    if (project == null)
    {
        Console.WriteLine("Project cannot be null");
```

```
        throw new ArgumentNullException("project");
    }
    foreach (Device device in project.Devices)
    //This example looks for devices located directly in the
project.
    //Devices which are stored in a subfolder of the project
will not be affected by this example.
    {
        foreach (DeviceItem deviceItem in device.DeviceItems)
        {
            DeviceItem deviceItemToGetService = deviceItem as
DeviceItem;
            SoftwareContainer container =
deviceItemToGetService.GetService<SoftwareContainer>();
            if (container != null)
            {
                HmiTarget hmi = container.Software as
HmiTarget;
                if (hmi != null)
                {
                    return hmi;
                }
            }
        }
    }
    return null;
}
//Deletes a single screen from a user folder or a system
folder
private static void DeleteScreenFromFolder(HmiTarget
hmiTarget)
{
    string screenName = "MyScreen";
```

```
    ScreenUserFolder folder =
hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
}
```

2.6 Example program

Application example: Creating API access in an application

The following program code shows basic API access in a Console application.

See also: Connecting to the TIA Portal

Note

Enabling the assembly reference property “Copy Local” is not supported. “Copy Local: True” means copying the references to Siemens.Engineering assemblies to the output directory of the application.

Remedy: Set “Copy Local: False” for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.




```
// Assembly reference: Siemens.Engineering.Base.dll
using Siemens.Engineering;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;

// Assembly reference: Siemens.Engineering.Step7.dll
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Units;

// System
using System;
using System.Collections.Generic;
```

```
using System.IO;
using System.Reflection;

namespace HelloOpenness
{
internal static class Program
{
private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

static Program()
{
// Register assembly resolver at the earliest stage in static
Program constructor
AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);
```

```
return loadedAssembly;
}

public static void Main()
{
Console.WriteLine("Please wait...");
try
{
HelloOpennessApp app = new HelloOpennessApp();
app.Run();
Console.WriteLine("Done");
}
catch (Exception exception)
{
Console.WriteLine("Error");
Console.WriteLine(exception);
}
Console.ReadLine();
}
}

internal class HelloOpennessApp
{
private const string ProjectsPath = @"C:\Projects";

public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
using (ExclusiveAccess exclusiveAccess =
portal.ExclusiveAccess("Generator is working..."))
{
Project project = portal.Projects.Create(new
DirectoryInfo(ProjectsPath), "Project_1");
Console.WriteLine($"Project has been created:
{project.Path.FullName}");

Device station = project.Devices.CreateWithItem(
"OrderNumber:6ES7 516-3AP03-0AB0/V4.0", "PLC_1", "S7-1500
Station_1"
);
```

```
Console.WriteLine($"Device has been created:
{station.Name}");

PlcSoftware plc = GetPlcSoftware(station);
if (plc == null)
{
Console.WriteLine("No PLC software found in the device.");
}
else
{
PlcUnitProvider softwareUnits =
plc.GetService<PlcUnitProvider>();
if (softwareUnits == null)
{
Console.WriteLine("Software units are not supported for this
PLC or firmware version.");
}
else
{
PlcUnit softwareUnit =
softwareUnits.UnitGroup.Units.Create("Software unit_1");
Console.WriteLine($"Software unit has been created:
{softwareUnit.Name}");
}
}

ICompilable compiler = station.GetService<ICompilable>();
if (compiler == null)
{
Console.WriteLine("Compile service is not available for this
device.");
}
else
{
CompilerResult result = compiler.Compile();
Console.WriteLine($"Compile: {result.WarningCount} warnings,
{result.ErrorCount} errors.");
}

project.Save();
project.Close();
```

```
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);

return loadedAssembly;
}

public static void Main()
{
Console.WriteLine("Please wait...");
try
{
HelloOpennessApp app = new HelloOpennessApp();
app.Run();
Console.WriteLine("Done");
}
catch (Exception exception)
{
Console.WriteLine("Error");
```

```
Console.WriteLine(exception);
}
Console.ReadLine();
}
}

internal class HelloOpennessApp
{
private const string ProjectsPath = @"C:\Projects";

public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
using (ExclusiveAccess exclusiveAccess =
portal.ExclusiveAccess("Generator is working..."))
{
Project project = portal.Projects.Create(new
DirectoryInfo(ProjectsPath), "Project_1");
Console.WriteLine($"Project has been created:
{project.Path.FullName}");

Device station = project.Devices.CreateWithItem(
"OrderNumber:6ES7 516-3AP03-0AB0/V4.0", "PLC_1", "S7-1500
Station_1"
);
Console.WriteLine($"Device has been created:
{station.Name}");

PlcSoftware plc = GetPlcSoftware(station);
if (plc == null)
{
Console.WriteLine("No PLC software found in the device.");
}
else
{
PlcUnitProvider softwareUnits =
plc.GetService<PlcUnitProvider>();
if (softwareUnits == null)
{
```

```
Console.WriteLine("Software units are not supported for this
PLC or firmware version.");
}
else
{
PlcUnit softwareUnit =
softwareUnits.UnitGroup.Units.Create("Software unit_1");
Console.WriteLine($"Software unit has been created:
{softwareUnit.Name}");
}
}

ICompilable compiler = station.GetService<ICompilable>();
if (compiler == null)
{
Console.WriteLine("Compile service is not available for this
device.");
}
else
{
CompilerResult result = compiler.Compile();
Console.WriteLine($"Compile: {result.WarningCount} warnings,
{result.ErrorCount} errors.");
}

project.Save();
project.Close();
}
}
}

private static PlcSoftware GetPlcSoftware(HardwareObject
hardwareObject)
{
// Breadth-first traversal of a station
Queue<HardwareObject> queue = new Queue<HardwareObject>();
queue.Enqueue(hardwareObject);

while (queue.Count > 0)
{
// Check all modules at the current level
```

```
foreach (DeviceItem deviceItem in queue.Dequeue().Items)
{
if (!deviceItem.IsPlugged) continue; // Skip if module is in
parking lot

if
(deviceItem.Classification.HasFlag(DeviceItemClassifications.CPU)
&&
deviceItem.GetService<SoftwareContainer>()?.Software is
PlcSoftware plcSoftware)
return plcSoftware;

// Enqueue all child modules for next level processing
queue.Enqueue(deviceItem);
}
}

return null;
}
}
}
```

2.7 Programming steps

Overview

TIA Portal Openness requires the following programming steps for access by means of the TIA Portal Openness API:

1. Make the TIA Portal known in the development environment
2. Set up program access to the TIA Portal
3. Activate program access to the TIA Portal
4. Publish and start the TIA Portal
5. Open project
6. Execute commands
7. Save and close the project
8. Terminate the connection to the TIA Portal

Note

Permitted strings

Only certain characters are allowed in strings in the TIA Portal. All strings passed to the TIA Portal via the TIA Portal Openness application are subject to these rules. If you pass an invalid character in a parameter, an exception is thrown.

See also

Example program
Use of the code examples

2.8 Connecting to the TIA Portal

Introduction

TIA Portal Openness has the possibilities to

- get running TIA Portal processes and attach to TIA Portal instances
- start new TIA Portal instances, either with or without graphical user interface

Note

Since you always use TIA Portal Openness by loading Siemens.Engineering assemblies from a specific TIA Portal version, the returned processes are all from that same TIA Portal version. It is not supported to access different versions in the same AppDomain.

The app.config file and the AppDomain.CurrentDomain.AssemblyResolve event are two solutions for resolving the Siemens.Engineering assemblies during runtime.

Note

Resolving Siemens.Engineering assemblies is only needed for TIA Portal Openness applications, but not for TIA Portal Add-Ins.

Using the app.config file

Set “Copy Local: False” for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.

By adding an application configuration file (app.config file) to a .NET project, you can customize how the Microsoft CLR locates and loads assemblies. The app.config file is deployed next to the executable and renamed to match the executable.

Note

The disadvantage of the app.config file is that it must contain the correct entries before the application starts. Changes cannot be made during program execution.

Example app.config file for .NET Framework 4.8:



```
Copies the following program code to clipboard.

<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<startup>
<supportedRuntime version="v4.0"
sku=".NETFramework,Version=v4.8" />
</startup>
<runtime>
<assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
<dependentAssembly>
<assemblyIdentity name="Siemens.Engineering.Base"
culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b " />
<!-- Edit the following version and path according to your
required TIA Portal Openness version -->
<codeBase version="xx.xx.xx.xx"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Base.dll" />
</dependentAssembly>
<!-- Depending on your application, you may need to provide
additional Siemens.Engineering assemblies -->
<dependentAssembly>
<assemblyIdentity name="Siemens.Engineering.Step7"
culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b" />
<!-- Edit the following version and path according to your
required TIA Portal Openness version -->
<codeBase version="xx.xx.xx.xx"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Step7.dll" />
</dependentAssembly>
</assemblyBinding>
</runtime>
</configuration>
```

Using AppDomain.CurrentDomain.AssemblyResolve event

The assembly resolver must be registered at the earliest stage in the program.

Register the AssemblyResolve event before using any TIA Portal Openness types. This cannot happen in the same method. Place all TIA Portal Openness code in a separate method or separate class that is called only after event subscription.

Note: The .NET runtime will throw an exception if it encounters TIA Portal Openness types before the resolver is registered.

Important: This applies not only to method bodies but also to:

- Class properties with TIA Portal Openness types
- Method parameters with TIA Portal Openness types
- Return values with TIA Portal Openness types

Even when placed in separate methods, these will be resolved during class loading.

When you move your TIA Portal Openness code outside the Program class, the resolver will be active before that classes are evaluated during runtime.

Example of recommended application code structure:

 Copies the following program code to clipboard.

```
internal static class Program
{
static Program()
{
// First register assembly resolver at the earliest stage in
static Program constructor
AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
}

public static void Main()
{
// Then call method with TIA Portal Openness code in separate
class
HelloOpennessApp app = new HelloOpennessApp();
app.Run();
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
// Code is shown in the next section
// ...
}
```

```
internal static class HelloOpennessApp
{
public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
// Do something with TIA Portal Openness
}
}
}
```

Example for a basic AssemblyResolve implementation where the installation path is given:



```
Copies the following program code to clipboard.

private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);
```

```
return loadedAssembly;
}
```

Note

These examples have demonstrated how to resolve multiple Siemens.Engineering assemblies using a robust and reliable approach. If your application architecture is very modular and your first class only accesses the TiaPortal class from Siemens.Engineering.Base.dll, it is sufficient to resolve only the Siemens.Engineering.Base.dll in your code. All subsequent Siemens.Engineering assemblies will then be automatically resolved by the Base library.

Note

Recommended way is using the TIA Portal Openness registry keys with the AppDomain.CurrentDomain.AssemblyResolve event.

See also: Information about installed TIA Portal Openness versions

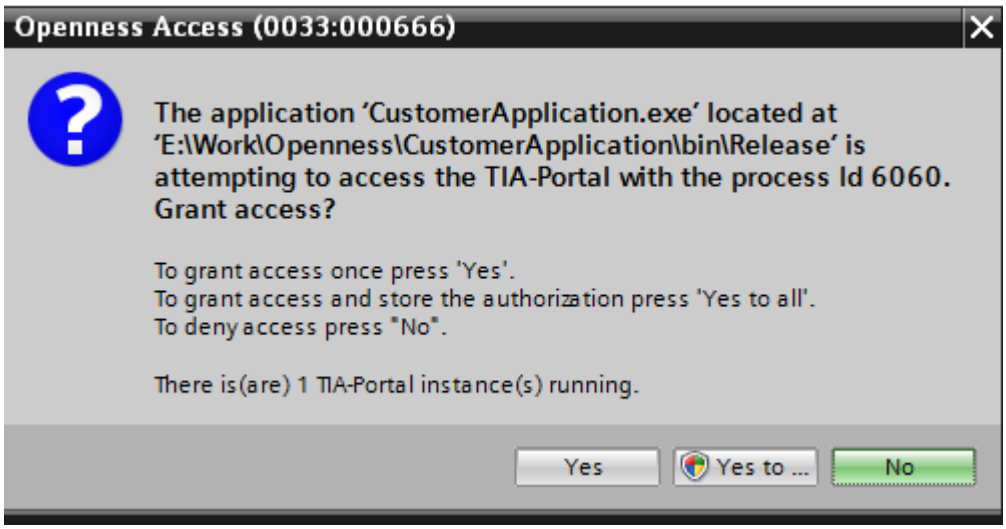
See also

TIA Portal Openness firewall

2.9 TIA Portal Openness firewall

TIA Portal Openness firewall prompt

When you try to connect to a running TIA Portal instance (includes starting a new TIA Portal instance) via TIA Portal Openness, TIA Portal will prompt you to accept or reject the connection like the following.



Note

Additionally, and beforehand, administrators must grant individual users the execution of any TIA Portal Openness application in general. This is managed by a local Windows group membership ("Siemens TIA Openness").

See also: Adding users to the "Siemens TIA Openness" user group

Allow connection to the TIA Portal once

Button "Yes": Users can allow individual TIA Portal Openness applications access to a specific (currently active) TIA Portal instance temporarily.

The next time the TIA Portal Openness application tries to connect to a TIA Portal instance, the firewall prompt will be shown again.

Addition of an AllowList entry by connecting to TIA Portal

Button "Yes to all": Administrators can allow individual TIA Portal Openness applications access to every TIA Portal instance permanently. This is managed by an automatically created Windows registry entry.

Note

If the file path, the modification date or the file content (file hash) of the TIA Portal Openness application changes, the application will be identified as a different one and the firewall prompt will be shown again.

Addition of an AllowList entry without using TIA Portal

Administrators can roll out a custom Windows registry file to automatically create an AllowList entry:



```
Copies the following program code to clipboard.

Windows Registry Editor Version 5.00
[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList\Custo
[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList\Custo
"Path"="C:\\Program Files\\CustomerApplication\\bin\\CustomerApplicatio
"DateModified"="2025/06/10 15:09:44.406"
"FileHash"="0rXRKUCNzMWHOMFrT52OwXzqJef10ran4UykTeBraaY="
```

The following example shows how you can calculate the values programmatically:



```
Copies the following program code to clipboard.

using System;
using System.Globalization;
```

```
using System.IO;
using System.Security.Cryptography;

// Usage:
// CalculateValues(new FileInfo("C:\\Program
Files\\CustomerApplication\\bin\\CustomerApplication.exe"));

public static void CalculateValues(FileInfo
opennessApplicationInfo)
{
using (FileStream stream =
opennessApplicationInfo.OpenRead())
{
HashAlgorithm hashAlgorithm = SHA256.Create();

using (hashAlgorithm)
{
byte[] computedHash = hashAlgorithm.ComputeHash(stream);
DateTime lastWriteTimeUtc =
opennessApplicationInfo.LastWriteTimeUtc;

// This is how the path should be represented
string path = opennessApplicationInfo.FullName;

// This is how the date modified should be formatted
string dateModified = lastWriteTimeUtc.ToString("yyyy/MM/dd
HH:mm:ss.fff", CultureInfo.InvariantCulture);

// This is how the file hash should be represented
string fileHash =
System.Convert.ToBase64String(computedHash);
}
}
}
```

See also: Information about installed TIA Portal Openness versions

2.10 Event handlers

Event handlers in TIA Portal Openness application

An instance of the TIA Portal provides the following events to which you can react with an event handler in a TIA Portal Openness application. You can access the attributes of notifications and

define the responses accordingly.

Event	Response
Disposed	Use this event to respond to the closing of the TIA Portal with a TIA Portal Openness application.
Notification	Use this event to respond to notifications of the TIA Portal with a TIA Portal Openness application. Notifications require only an acknowledgment, e.g. "OK".
Confirmation	Use this event to respond to confirmations of the TIA Portal with a TIA Portal Openness application. Confirmations always require a decision, e.g. "Do you want to save the project?".

Program code

Modify the following program code to register event handlers in a TIA Portal Openness application:

 Copies the following program code to clipboard.

```
//Register event handler for Disposed-Event
    ....
        tiaPortal.Disposed +=TiaPortal_Disposed;
....
private static void TiaPortal_Disposed(object sender,
EventArgs e)
{
    ....
}
//Register event handler for Notification-Event
    ....
        tiaPortal.Notification += TiaPortal_Notification;
....
private static void TiaPortal_Notification(object sender,
NotificationEventArgs e)
{
    ....
```

```
}
//Register event handler for Confirmation-Event
    ....
        tiaPortal.Confirmation += TiaPortal_Confirmation;
....
private static void TiaPortal_Confirmation(object sender,
ConfirmationEventArgs e)
{
    ....
}
```

Attributes of TIA Portal notifications

TIA Portal notifications have the following attributes:

Attribute	Description
`Caption`	Returns the name of the confirmation.
`DetailText`	Returns the detail text of the confirmation.
`Icon`	Returns the icon of the confirmation.
`IsHandled`	Returns the confirmation or specifies if it is still pending.
`Text`	Returns the text of the confirmation.

Attributes of confirmations

Confirmations have the following attributes:

Attribute	Description
`Caption`	Returns the name of the confirmation.
`Choices`	Returns the option to acknowledge the confirmation.
`DetailText`	Returns the detail text of the confirmation.

`Icon`	Returns the icon of the confirmation.
`IsHandled`	Returns the confirmation or specifies if it is still pending.
`Result`	Returns the result of the acknowledgment or specifies it.
`Text`	Returns the text of the confirmation.

See also

Program-controlled acknowledgement of dialogs with system events

2.11 Program-controlled acknowledgement of dialogs with system events

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- Event handlers are registered.
  See Connecting to the TIA Portal

Application

When you operate the TIA Portal with the user interface, dialogs with system events are displayed for some program sequences. You decide how you want to proceed based on these system events.

When the TIA Portal is accessed with a TIA Portal Openness application, these system events must be acknowledged by means of corresponding ".NET" events.

The permitted confirmations are contained in the `Choices` list:

- `Abort`
- `Cancel`
- `Ignore`
- `No`
- `NoToAll`
- `None`
- `OK`
- `Retry`
- `Yes`

- `YesToAll`

The value of `ConfirmationEventArgs.Result` must be one of the above-mentioned entries. Otherwise, an exception is thrown.

Program code

Modify the following program code to notify the project engineer about executed actions of a TIA Portal Openness application:



```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
Namespace ConfirmDialogsWithSystemMessagesProgrammatically
{
```

```
class internal program()
{
private static void Main(string[] args)
{
// To respond to a confirmation event
using (TiaPortal tiaPortal = new TiaPortal());
{
tiaPortal.Confirmation += TiaPortalConfirmation(object
sender, ConfirmationEventArgs e);
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
}
private void TiaPortalConfirmation(object sender,
ConfirmationEventArgs e)
{
    ...
}
//Handles notifications
using (TiaPortal tiaPortal = new TiaPortal())
{
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
```

```
finally
{
tiaPortal.Notification -= Notification;
}
}
}
}
```

2.12 Terminating the connection to the TIA Portal

Introduction

If you started the TIA Portal instance without a user interface and if your application is the only TIA Portal Openness client attached to the TIA Portal, you can close the TIA Portal instance with the TIA Portal Openness application. Otherwise, you disconnect the TIA Portal Openness application from the TIA Portal instance.

Use the `IDisposable.Dispose()` method to separate or close the active instance of the TIA Portal.

You can use the `IDisposable.Dispose()` method as follows:

- With a using statement.
- Surround the object description with a try-finally block and call the `IDispose.Dispose()` method within the finally block.

You can no longer access the TIA Portal when you close the active instance of the TIA Portal.

Note

When a configuration engineer closes the TIA Portal instance despite ongoing access of a TIA Portal Openness application, an exception of the class "NonRecoverableException" is thrown in the TIA Portal Openness application on the next API access. You can subscribe to the dispose event to get a call when the TIA Portal is closed.

Program code

Modify the following program code to separate or close the connection to the TIA Portal:

```
 Copies the following program code to clipboard.

// Add code to dispose the application if the application is
still instantiated
```

```
if (tiaPortal != null)
{
    tiaPortal.Dispose();
}
```

See also

Event handlers

2.13 Diagnostic interfaces on TIA Portal

Application

You can retrieve certain diagnostic information from running instances of TIA Portal via a static method. The diagnostic interface is implemented on the TiaPortalProcess object, which can be retrieved for any currently running instance of the TIA Portal.

The diagnostic interface is not blocking, so you can retrieve the TiaPortalProcess object and access it's members, regardless of if the TIA Portal is busy or not. The diagnostic interface includes the following Members:

Class TiaPortalProcess

Member	Type	Function
AcquisitionTime	DateTime	The time when the TiaPortalProcess object was acquired. Since the TiaPortalProcess object represents a completely static snapshot of the state of the TIA Portal at a given point in time, the information it contains may become outdated.
Attach	TiaPortal	Attaches to the given TiaPortalProcess, it returns a TiaPortal instance.
AttachedSessions	IList<TiaPortalSession>	A collection of all other sessions currently attached to the same TIA Portal. This collection can be empty. Each session is represented

		by a TiaPortalSession object.
Attaching	EventHandler<AttachingEventArgs>	This event enables an application to approve any attempts to attach to the TIA Portal. When another application attempts to attach to the TIA Portal, the subscribers of this event are notified and given 10 seconds to approve the attachment. If any subscriber ignores this event or does not respond in time, it is understood to be denying the other application permission to attach. Crashed applications, being unable to respond to this event and cannot cause an application to be denied permission to attach.
Dispose	void	Closes the associated TIA Portal instance.
Id	int	The Process ID of the TIA Portal.
InstalledSoftware	IList<TiaPortalProduct>	A collection of all the products currently installed as part of the TIA Portal. Each product is represented by a TiaPortalProduct object, which is described below.
Mode	TiaPortalMode	The mode in which the TIA Portal was started. The current values are WithUserInterface and WithoutUserInterface.
Path	FileInfo	The path to the executable of the TIA Portal.

ProjectPath	FileInfo	The path to the project which is currently open in the TIA Portal. If no project is open, this attribute will be null.

Class TiaPortalSession

Member	Type	Function
AccessLevel	TiaPortalAccessLevel	The level access of the session. It is represented as a flags enum, where multiple levels of access are possible. TiaPortalAccessLevel is described in detail below.
AttachTime	DateTime	The time when the connection to the TIA Portal was established.
Id	int	The Id of the current session.
IsActive	bool	Returns "true" if the TIA Portal is currently processing a call from the running session.
Dispose	void	Severs the process's connection to the TIA Portal. This method does not kill the process itself in the way that System.Diagnostics.Process.Kill would do. The application whose connection is terminated will still get a disposed event, but there is no other indication of why the connection was terminated.
ProcessId	int	The process ID of the attached process.
ProcessPath	FileInfo	The path to the executable of the attached process.
TrustAuthority	TiaPortalTrustAuthority	Indicates if the current session was started by a process that was signed, and if it is a TIA

		Portal Openness certificate or not. TrustAuthority is a flags enum and is described below.
UtilizationTime	TimeSpan	The period of time the process has spent actively using the TIA Portal.Combined with the AttachTime attribute, this could be used to determine usage percentages or similar data.
Version	string	The version of TIA Portal Openness to which the session is attached to.

Enum TiaPortalAccessLevel

Enum Value	Function
None	This is not a valid value. It is included because TiaPortalAccessLevel is a flags enum which needs an appropriate "zero value" to represent no flags being set, but it will never appear in actual use because no session can be started that has no access.
Published	The session has access to published functionality.
Modify	The session has modify access.

Enum TiaPortalTrustAuthority

Enum Value	Function
None	The main module of the attached process is not signed with a certificate.
Signed	The main module is signed with a certificate, which is not a TIA Portal Openness certificate.
Certified	The main module is signed with a TIA Portal Openness certificate.

CertifiedWithExpiration	The main module is signed with a TIA Portal Openness certificate that will become invalid at the end of its lifetime.

Class TiaPortalProduct

Member	Type	Function
Name	string	The name of the product (e.g. STEP 7 Professional).
Options	IList<TiaPortalProduct>	A collection of all optional packages that belong to the connected TIA Portal, represented as TiaPortalProduct objects. If an option package itself has option packages, this nesting could continue.
Version	string	The version string of the product.

The following code snippet provides an example of how to use the diagnostic Interface to query information and of how to use them in your application.



```
Copies the following program code to clipboard.
public void TiaPortalDiagnostics()
{
    IList<TiaPortalProcess> tiaPortalProcesses =
TiaPortal.GetProcesses();
    foreach (TiaPortalProcess tiaPortalProcess in
tiaPortalProcesses)
    {
        Console.WriteLine("Process ID: {0}",
tiaPortalProcess.Id);
        Console.WriteLine("Path: {0}",
tiaPortalProcess.Path);
        Console.WriteLine("Project: {0}",
tiaPortalProcess.ProjectPath);
```

```
        Console.WriteLine("Installed Software:");
        EnumerateInstalledProducts(tiaPortalProcess.InstalledSoftware);
        Console.WriteLine("Attached Openness Applications:");
        foreach (TiaPortalSession session in
tiaPortalProcess.AttachedSessions)
        {
            Console.WriteLine("Process: {0}", session.ProcessPath);
            Console.WriteLine("Process ID: {0}", session.ProcessId);
            DateTime attachTime = session.AttachTime;
            TimeSpan timeSpentAttached = DateTime.Now - attachTime;
            TimeSpan utilizationTime = session.UtilizationTime;
            long percentageTimeUsed = (utilizationTime.Ticks /
timeSpentAttached.Ticks) * 100;
            Console.WriteLine("AttachTime: {0}", attachTime);
            Console.WriteLine("Utilization Time: {0}",
utilizationTime);
            Console.WriteLine("Time spent attached: {0}",
timeSpentAttached);
            Console.WriteLine("Percentage of attached time spent using
TIA Portal: {0}", percentageTimeUsed);
            Console.WriteLine("AccessLevel: {0}", session.AccessLevel);
            Console.WriteLine("TrustAuthority: {0}",
session.TrustAuthority);
            if ((session.TrustAuthority &
TiaPortalTrustAuthority.Certified) !=
TiaPortalTrustAuthority.Certified)
            {
                Console.WriteLine("TrustAuthority doesn't match
required level, attempting to terminate connection to TIA Portal.");
                session.Dispose();
            }
        }
    }
```

```
}
public void
EnumerateInstalledProducts(IEnumerable<TiaPortalProduct>
products)
{
    foreach (TiaPortalProduct product in products)
    {
        Console.WriteLine("Name: {0}", product.Name);
        Console.WriteLine("Version: {0}", product.Version);
        //recursively enumerate all option packages
        Console.WriteLine("Option Packages \n:");
        EnumerateInstalledProducts(product.Options);
    }
}
```

Security Relevant Information

Because of the fact that no connection to the TIA Portal is needed to use the diagnostics interface, it's possible to write a Windows service that uses the attaching event to check any application attempting to attach to a TIA Portal, e.g. only applications that begin with your company's name are allowed to attach. Another option might be to always grant access, but write information about attaching processes to a log. The following program code is an example event handler to check incoming connections:

```
 Copies the following program code to clipboard.
public void OnAttaching(object sender, AttachingEventArgs e)
{
    string name =
Path.GetFileNameWithoutExtension(e.ProcessPath);
    TiaPortalAccessLevel requestedAccessLevel = e.AccessLevel
& TiaPortalAccessLevel.Published;
    TiaPortalTrustAuthority certificateStatus =
e.TrustAuthority &TiaPortalTrustAuthority.Certified;
    if (requestedAccessLevel ==
TiaPortalAccessLevel.Published &&
       certificateStatus == TiaPortalTrustAuthority.Certified
&&
```

```
        name.StartsWith("SampleCustomerName"))
    {
        e.GrantAccess();
    }
}
```

2.14 Exclusive access

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

The "TIA Portal" class provides the method "ExclusiveAccess(String text)" to establish an exclusive access to an attached TIA Portal process. The usage of an exclusive access is highly recommended even if it is not mandatory.

Use "ExclusiveAccess" in an "using" statement to ensure it is disposed attribute even when exceptions occur or the application is shutdown.

Note

Any attempt to create a second exclusive access within the scope of an open exclusive access will result in an recoverable exception being raised.

Modify the following example to get "ExclusiveAccess" to an instance:

```
 Copies the following program code to clipboard.
...
[assembly: AssemblyTitle("MyApplication")]
// This will be used for the exclusive access dialog when present....
TiaPortal tiaPortal = ...;
using (ExclusiveAccess exclusiveAccess = tiaPortal.ExclusiveAccess("My Activity"))
{
```

```
    ...
}
```

After acquiring an "ExclusiveAccess" instance for a given TIA Portal process a dialog will be displayed. This dialog will display the message provided during instantiation. In addition the following information of the client application will be displayed:

- the assembly title of the manifest data if available; otherwise, the process name
- the process ID
- the SID (Openness Session ID of the client application's TiaPortal instance)

Note

There can be multiple sessions active for a given TIA Portal Openness client application because there can be multiple instances of TiaPortal each associated with the same TIA Portal process.

The client application can also update the displayed content of the exclusive access dialog by setting the "Text" attribute with new values. Modify the following program code to invoke this behavior:



```
Copies the following program code to clipboard.
exclusiveAccess = ...;
...
exclusiveAccess.Text = "My Activity Phase 1";
...
exclusiveAccess.Text = "My Activity Phase 2";
...
exclusiveAccess.Text = String.Empty; // or null;
...
```

You can request that the exclusive access could be cancelled by selecting the "Cancel" button. Modify the following program code to invoke this behavior:



```
Copies the following program code to clipboard.
exclusiveAccess = ...;
...
if (exclusiveAccess.IsCancellationRequested)
{
    // stop your activity
```

```
    ...
}
else
{
    // continue your activity
    ...
}
...
```

2.15 Dynamic behaviours

Dynamic behaviors in Openness

The Openness users can determine objects and their attributes during the run time. They also are able to invoke actions and navigate to related objects bidirectionally in the hierarchy. In order to support dynamic behaviors the following interfaces are provided and implemented by Openness objects:

IEngineeringInstance

The interface that allows up navigation in the Object Model.

Properties:

- IEngineeringObject Parent - Parent object of an instance.

IEngineeringCompositionOrObject

The interface is implemented by both engineering objects and compositions.

IEngineeringObject

The interface is implemented by the majority of objects that are available for the Openness users. IEngineeringObject interface is derived from IEngineeringCompositionOrObject and IEngineeringInstance:

Methods:

- IList<EngineeringCreationInfo> GetCreationInfos(string compositionName) - Gets the list of composition infos available for the object.
- IEngineeringObject Create(string compositionName, Type type, IEnumerable<KeyValuePair<string, object>> parameters) - Creates an IEngineeringObject of indicated type initialized with values as indicated compositionName within the parameters
- object GetAttribute(string name) - Gets an attribute with the given name.

- IList<EngineeringAttributeInfo> GetAttributeInfos() - Returns a collection of EngineeringAttributeInfo objects describing the different attributes on this object.
- IList<object> GetAttributes(IEnumerable<string> names) - Gets a list of attributes for the given names
- IEngineeringCompositionOrObject GetComposition(string name) - Gets an IEngineeringCompositionOrObject with the given name.
- IList<EngineeringInvocationInfo> GetInvocationInfos() - Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object.
- object Invoke(string name, IEnumerable<KeyValuePair<Type, object>> parameters) - Invokes the method represented by the current instance, using the specified parameters.
- void SetAttribute(string name, object value) - Sets an attribute with the given name to the given value
- void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes) - Sets the attributes with the given names to the given values as indicated in attributes.
- EngineeringObjectHandle GetHandle() - Gets the object's unique handle.
- void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler) - Sets the attributes with the given names to the given values as indicated in attribute key value pair. If any error occurs (such as, attribute name/value is invalid), a callback call will be triggered so that you can decide whether to ignore and continue with setting value to the next attribute or abort setting the values to the remaining attributes.

The SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler) is supported by the following Engineering objects:

- Siemens.Engineering.HW.Features.NetworkInterface
- Siemens.Engineering.HW.Features.NetworkPort
- Siemens.Engineering.HW.Features.PlcAccessLevelProvider
- Siemens.Engineering.HW.Address
- Siemens.Engineering.HW.Channel
- Siemens.Engineering.HW.Device
- Siemens.Engineering.HW.DeviceImpl
- Siemens.Engineering.HW.DeviceItem
- Siemens.Engineering.HW.DeviceItemImpl
- Siemens.Engineering.HW.HardwareObject
- Siemens.Engineering.HW.IoConnector
- Siemens.Engineering.HW.IoController
- Siemens.Engineering.HW.IoSystem
- Siemens.Engineering.HW.MrpDomain

- Siemens.Engineering.HW.Node
- Siemens.Engineering.HW.Subnet
- Siemens.Engineering.HW.SyncDomain
- Siemens.Engineering.HW.TransferArea

If any other Engineering object is used, the method behaves similar to the method void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes) which has no callback.

- GetAttributes(Siemens.Engineering.AttributeAccessOption) - Get all attributes of an engineering object filtered by access mode with a single call. You can use the flag enum Siemens.Engineering.AttributeAccessOptions as a parameter for bulk API GetAttributes on an Engineering object to filter attributes based on access modes. The below options are available under AttributeAccessOptions:
  - None: No Attributes will be retrieved with this option. Empty list will be returned if this option is used in GetAttributes API.
  - ReadOnly: Attributes which have only read access.
  - WriteOnly: Attributes which have only write access. An exception of type Siemens.Engineering.EngineeringNotSupported Exception will be thrown If there is at-least one write only attribute on an engineering object and user invokes GetAttributes with AttributeAccessOptions.WriteOnly option.
  - ReadWrite: Attributes which have read and write access.

Program code: SetAttributes(IEnumerable<KeyValuePair<string, object>>) method



```
Copies the following program code to clipboard.

...

...

List<KeyValuePair<string, object>> attributeKeyValuePair =
new List<KeyValuePair<string, object>>()

{

new KeyValuePair<string, object>("Name", "DeviceItemName")
};

deviceItem.SetAttributes(attributeKeyValuePair,
(AttributeConfiguration attributeConfiguration) =>
{

//This callback delegate will be invoked when there is an
error while executing the SetAttributes API,
```

```
//reason could be either due to the attribute having invalid
attribute name, invalid value,
//invalid type of the value or when tried to set the value to
a readonly attribute
switch (attributeConfiguration)
{
case AttributeReadOnly attributeReadOnly:
attributeReadOnly.CurrentSelection =
AttributeChoiceSelection.Ignore;
break;
case AttributeNameUnsupported attributeNameUnsupported:
//Attribute name is not supported
attributeNameUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore;
// Or AttributeChoiceSelection.Abort
break;
case AttributeTypeUnsupported attributeTypeUnsupported:
//The the type of supplied value is not supported
attributeTypeUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore; // Or
AttributeChoiceSelection.Abort
break;
case AttributeValueUnsupported attributeValueUnsupported:
//The value is not supported
attributeValueUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore; // Or
AttributeChoiceSelection.Abort
break;
}
}
);
```

Program code:
GetAttributes(Siemens.Engineering.AttributeAccessOption)



```
private void GetAllAttributes(ProjectBase project)
{
    DeviceItem deviceItem = project.Devices[0].Items[0];
    // If there is no readonly attribute available in
DeviceItem
    // then the returned list will be empty.
    IReadOnlyList<KeyValuePair<string, object>>
attributeNameValuePairs =
deviceItem.GetAttributes(AttributeAccessOptions.ReadOnly);
    foreach(KeyValuePair<string, object> pair in
attributeNameValuePairs)
    {
    Console.WriteLine("Attribute name: {0}, value: {1}",
pair.Key, pair.Value);
    }
}
```

IEngineeringRoot

This interface is implemented by TiaPortal objects and indicates that the user is at the top object in the hierarchy. IEngineeringRoot is derived from IEngineeringObject, IEngineeringCompositionOrObject and IEngineeringInstance.

Methods:

- IEngineeringObject GetObject(EngineeringObjectHandle objectHandle) - Gets the object from a handle

IEngineeringComposition

The interface is implemented by the majority of objects that are available for the Openness users. IEngineeringObject interface is derived from IEngineeringCompositionOrObject and IEngineeringInstance. In addition to the properties and methods described in Working with compositions, the following methods are supported.

Methods:

- IEngineeringObject Create(Type type, IEnumerable<KeyValuePair<string, object>> parameters) - Creates an IEngineeringObject of indicated type initialized with values as indicated in parameters..
- IList<EngineeringCreationInfo> GetCreationInfos() - Gets the collection of EngineeringCreateInfo objects describing the different CreateInfos on this object.

- IList<EngineeringInvocationInfo> GetInvocationInfos() - Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object.
- object Invoke(string name, IEnumerable<KeyValuePair<Type, object>> parameters) - Invokes the method represented by the current instance, using the specified parameters.

IEngineeringObjectAssociation

This interface is implemented by the collection of associated engineering objects. This interface is derived from IEngineeringAssociation and IEngineeringInstance. See Working with associations

IEngineeringServiceProvider

The interface is implemented by engineering objects that can provide service(s).

Methods:

- T GetService<T>() where T : class, IEngineeringService - Gets an instance of type T.
- IList<EngineeringServiceInfo> GetServiceInfos() - Returns a collection of EngineeringServiceInfo objects describing the different services on this object.

IEngineeringService

The interface representing an engineering service that is provided by IEngineeringServiceProvider.

EngineeringObjectHandle

The structure that provides a unique object handle.

See also

Working with compositions
Working with associations

2.16 Working with associations

Accessing associations

An association describes the relationship between two or more objects at type level.

TIA Portal Openness supports access to associations via index and via "foreach" loops. Direct access, for example via string name, is not supported.

Attributes

The following attributes are available:

- `int Count`
- `bool IsReadonly`
- `IEngineeringObject Parent`
- `retType this [ int index ] { get; }`

Methods

TIA Portal Openness supports the following methods:

- `int IndexOf ( type )`: Returns the index in the association for a transferred instance.
- `bool Contains ( type )`: Determines whether the transferred instance is contained in the association.
- `IEnumerator GetEnumerator <retType>()`: Employed within "foreach" loops to access an object.
- `void Add ( type )` [1]: Adds the transferred instance to the association.
- `void Remove ( type )` [1]: Removes the transferred instance from the association.

[1]: Not supported by all associations.

2.17 Working with compositions

Accessing compositions

A composition is the special case of an association. A composition expresses a semantic relationship of two objects, of which one is part of the other.

Attributes

The following attributes are available:

- `int Count`
- `bool IsReadonly`
- `IEngineeringObject Parent`
- `retType this [int index] {get;}`: Indexed access to an object of the composition.

  This type of access should only be used in a targeted manner, as each indexed access operation exceeds process boundaries.

Methods

TIA Portal Openness supports the following methods:

- `retType Create (id, …)`: Creates a new instance and adds this instance to the composition.

The signature of the method depends on the way in which the instance is created. This method is not supported by all compositions.

- `type Find (id, …)`: Scans a composition for the instance with the transferred ID.

  The search is not recursive. The signature of the method depends on the way in which the instance is searched for. This method is not supported by all compositions.

- `IEnumerator GetEnumerator<retType> ()`: Employed within "foreach" loops to access an object.
- `Delete (type)` [1]: Deletes the instance specified by the current object reference.
- `int IndexOf (type)`: Returns the index in the composition for a transferred instance.
- `bool Contains (type)`: Determines whether the transferred instance is contained in the composition.
- `void Import(string path, ImportOptions importOptions)` [1]: Used for each composition that contains importable types.

  Each import signature includes a configuration parameter of the type "ImportOptions" ("None", "Overwrite") by which the user controls the import behavior.

[1]: Not supported by all compositions.

2.18 Verifying object equality

Application

As user of a TIA Portal Openness API, you can check that objects are the same with program code:

- You check whether two object references are the same with the operator "`==`".
- Use the `System.Object.Equals()` method to check if both objects are really identical with regard to the TIA Portal.

Program code

Modify the following program code to check for object reference types:

```
 Copies the following program code to clipboard.
...
//Composition
DeviceComposition sameCompA = project.Devices;
DeviceComposition sameCompB = project.Devices;
if (sameCompA.Equals(sameCompB))
{
```

```
    Console.WriteLine("sameCompA is equal to sameCompB");
}
if (!(sameCompA == sameCompB))
{
    Console.WriteLine("sameCompA is not reference equal to
sameCompB");
}
DeviceComposition sameCompAsA = sameCompA;
if (sameCompAsA.Equals(sameCompA))
{
    Console.WriteLine("sameCompAsA is equal to sameCompA");
}
if (sameCompAsA == sameCompA)
{
    Console.WriteLine("sameCompAsA is reference equal to
sameCompA");
}
MultiLingualGraphicComposition notSameComp =
project.Graphics;
if (!sameCompA.Equals(notSameComp))
{
    Console.WriteLine("sameCompA is not equal to
notSameComp");
}
```

2.19 Read operations for attributes

Group operations and standard read operations for attributes

TIA Portal Openness supports access to attributes via the following methods which are available at the object level:

- Group operation for read access
- Standard read operations

Program code for group operations

```
Copies the following program code to clipboard.
//Exercise GetAttributes and GetAttributeNames
//get all available attributes for a device,
//then get the names for those attributes, then display the
results.
private static void DynamicTest(Project project)
{
    Device device = project.Devices[0];
    IList<string> attributeNames = new List<string>();
    IList<EngineeringAttributeInfo> attributes =
((IEngineeringObject)device).GetAttributeInfos();
    foreach (EngineeringAttributeInfo
engineeringAttributeInfo in attributes)
    {
        string name = engineeringAttributeInfo.Name;
        attributeNames.Add(name);
    }
    IList<object> values =
((IEngineeringObject)device).GetAttributes(attributeNames);
    for (int i = 0; i < attributes.Count; i++)
    {
        Console.WriteLine("attribute name: " +
attributeNames[i] + " value: " + values[i]);
    }
}
```

Group operation for read access

This method is available for any object:

```
public abstract IList<object> GetAttributes(IEnumerable<string> names);
```

Standard read operations

The following operations are available:

- Retrieve the names of available attributes:

Use the method GetAttributeInfos() on an `IEngineeringObject`.

- Generic method for reading an attribute

  ```
  public abstract object GetAttribute(string name);
  ```

Note

Dynamic attributes are not shown in IntelliSense because their availability depends on the status of the object instance.

2.20 Transaction handling

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal

- A project is open.

  See Opening a project

Operation

A persistence (project, library, etc.) opened within an associated TIA Portal process can be modified by a TIA Portal Openness client application. You can produce this modification from a single operation or by a series of operations. Depending on the activity, it is reasonable to group these operations into a single undo unit for more logical workflows. Additionally there are performance advantages provided by grouping operations into a single undo unit. To support this, the "ExclusiveAccess" class provides the method "Transaction(ITransactionSupport persistence, string undoDescription)". The invocation of this method results in the instantiation of a new disposable object of type "Transaction". You have to provide a description of the transaction's contents (the text attribute cannot be null or Empty). While this instance has not been disposed, all client application operations will be grouped into a in a single undo unit within the associated TIA Portal process.

Modify the following program code to acquire a "Transaction" instance:

```
Copies the following program code to clipboard.
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
    ...
```

```
}
```

Note

Use a "using" statement to instatiate a "Transaction" to ensure it is disposed properly even when exceptions occur, thus rolling back the transaction.

Consistent commit or rollback

The use of a "Transaction" within a client application helps you to ensure that there is a predictable way to commit or rollback a set of modifications. Your client application must decide whether or not to commit their modifications to a persistence. To do this your application must request that the modifications within the scope of an open transaction be committed when the transaction is disposed by invoking the 'Transaction.CommitOnDispose()' method. If this method is never invoked in the code flow, the modifications within the scope of the open transaction will automatically be rolled back when it is disposed.

Note

Any call to Transaction.CommitOnDispose() made after a Siemens.Engineering.TargetInvocationException has occurred within an open transaction will result in a recoverable exception being thrown.

If an exception occur after making the request, all modifications within the scope of an open transaction will still be rolled back on its disposal.

There are two properties available to determine the committal state of a given transaction:

Property	Data type	Description
Transaction.CanCommit	boolean	Indicates if a prior call to Transaction.CommitOnDispose() will be honored, or if a subsequent call to Transaction.CommitOnDispose() will result in a recoverable exception
Transacation.CommitRequested	boolean	Indicates if a call to Transaction.CommitOnDispose() has been made by the user within the given transaction

Modify the following program code to create a single undo unit in the attached TIA Portal containing two "Create" modifications:



```
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
// Create a single undo unit with name "My Operation" in
attached TIA Portal
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation")
{
project.DeviceGroups.Create("My Group 1");
project.DeviceGroups.Create("My Group 2");
transaction.CommitOnDispose();
}
```

Modify the following program code to create no undo unit:

```
Copies the following program code to clipboard.
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 2");
}
```

Modify the following program code to create no undo unit in the attached TIA Portal. This is due to exception raised by the second attempt to create a folder "My Folder 1" even though the request to commit on disposal was made before the exception was raised:

```
Copies the following program code to clipboard.
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 2");
```

```
transaction.CommitOnDispose();
project.DeviceGroups.Create("My Folder 1");
}
```

Modify the following program code to create no undo unit in the attacted TIA Portal. This is due to excpetion raised by the second attempt to create a folder "My Folder 1" even though the try/catch block surrounds the call that caused the exception to be raised:

```
Copies the following program code to clipboard.
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 2");
transaction.CommitOnDispose();
try
{
project.DeviceGroups.Create("My Folder 1");
}
catch (Siemens.Engineering.TargetInvocationException ex)
{
...
}
}
```

Modify the following program code to create undo unit created in the attached TIA Portal. This is due to the invocation of Transaction.CommitOnDispose() after a Siemens.Engineering.TargetInvocationException is raised prior in the code stream even though a try/catch block surrounds the call that caused the exception to be raised:

```
Copies the following program code to clipboard.
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
```

```
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
try
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 1");
}
catch (Siemens.Engineering.TargetInvocationException ex)
{
...
}
transaction.CommitOnDispose();
}
```

Modify the following program code to have a transaction rolled back . This is due to the Transaction.CommitOnDispose() never being executed if the execution of any prior statement raises an exception within the scope of the given transaction.



```
Copies the following program code to clipboard.
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
try
{
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 1");
transaction.CommitOnDispose();
}
}
catch (Exception ex)
{
```

```
...
}
```

Restrictions

The following actions are not allowed inside of a transaction. Calling these will result in a recoverable exception:

- Compile
- Go Online
- Go Offline
- ProjectText Import
- ProjectText Export
- Open Global Library
- Close Global Library
- Project Create
- Project Open
- Project OpenWithUpgrade
- Project Save
- Project SaveAs
- Project Close
- Set value in a TIA Portal Setting

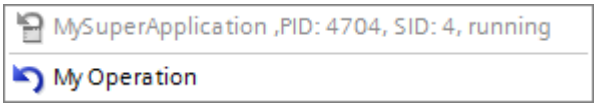
Undo behavior

Actions performed by a TIA Portal Openness client application can result in undo units within the attached TIA Portal process. Each of these undo entries will be grouped under a location entry. This location entry will compose the following information from the client application:

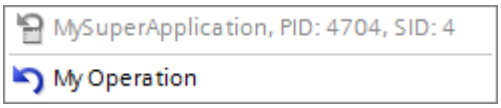
- the assembly title from the manifest data if available; otherwise, the process name
- the process ID
- the SID (Openness Session ID of the client application's TiaPortal instance)
- optionally an indication that the client process is still running

These entries will be one of the following two kinds:

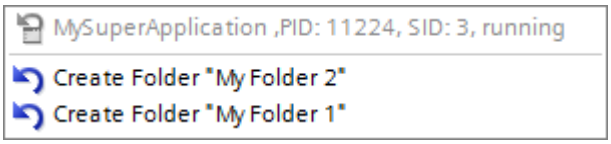
1. The operations that are gathered into one undo transaction as a result of using a "Transaction" have the description as provided by the client application when the "Transaction" was instantiated.
    - Undo entry for a running client application:



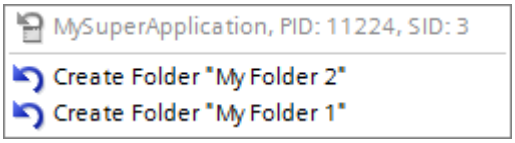
- Undo entry for a stopped client application:



2. The operations that are executed individually have individual undo entries describing the operation as defined in the respective command meta data.
   - Undo entry for a running client application:



   - Undo entry for a stopped client application:



2.21 Creating a DirectoryInfo/FileInfo object

Application

The instances of `DirectoryInfo` and `FileInfo` classes have to contain an absolute path. Otherwise the methods using the `DirectoryInfo` or `FileInfo` objects.will lead to an exception.

Program code

Modify the following program code to create a `DirectoryInfo` or a `FileInfo` object.

```
Copies the following program code to clipboard.
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
```

```
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
namespace CreateObjectDirectoryInfoFileInfo
{
internal class Program
{
private static void Main(string[] args)
{
RunTiaPortal();
//..
//Create a DirectoryInfo object
string directoryPath = @"D:\Test\Project 1";
DirectoryInfo directoryInfo = new
DirectoryInfo(directoryPath);
//Create a FileInfo object
//please adapt the path and the extension apx to the
installed version of TIA Portal
string fileName = @"D:\Test\Project 1\Project 1.apx";
FileInfo fileInfo = new FileInfo(fileName);
//...
```

```
}
private static void RunTiaPortal()
{
Console.WriteLine("Starting TIA Portal");
using (TiaPortal tiaPortal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
Console.WriteLine("TIA Portal has started");
ProjectComposition projects = tiaPortal.Projects;
Console.WriteLine("Opening Project...");
// please adapt the path and the extension apx to the
installed version of TIA Portal
FileInfo projectPath = new
FileInfo(@"C:\Demo\AnyCompanyProject.apx");
//edit the path according to your project
Project project = null;
try
{
project = projects.OpenWithUpgrade(projectPath);
}
catch (Exception)
{
Console.WriteLine(String.Format("Could not open project {0}",
projectPath.FullName));
Console.WriteLine("Demo complete hit enter to exit");
Console.ReadLine();
return;
}
Console.WriteLine(String.Format("Project {0} is open",
project.Path.FullName));
IterateThroughDevices(project);
project.Close();
```

```
Console.WriteLine("Demo complete hit enter to exit");
Console.ReadLine();
}
}
private static void IterateThroughDevices(Project project)
{
if (project == null)
{
Console.WriteLine("Project cannot be null");
return;
}
Console.WriteLine(String.Format("Iterate through {0}
device(s)", project.Devices.Count));
foreach (Device device in project.Devices)
{
Console.WriteLine(String.Format("Device: \"{0}\".",
device.Name));
}
Console.WriteLine();
}
}
}
```

2.22 Self-description support for attributes, navigators, actions, and services

Application

In TIA Portal Openness each IEngineeringServiceProvider of the TIA Portal Openness API describes its capabilities to potential calls.

Self-description Support on IEngineeringObject

Method Name	Return values
GetCompositionInfos	Returns a collection of EngineeringCompositionInfo objects

	describing the different compositions of these objects. EngineeringCompositionInfo is described below.
GetAttributeInfos	Returns a collection of EngineeringAttributeInfo objects describing the different attributes of these objects. EngineeringAttributeInfo is described below.
GetCreationInfos(string compositionName)	Returns a collection of EngineeringCreationInfo objects describing the different creation methods for items on a given composition of this object. EngineeringCreationInfo is described below.
GetInvocationInfos	Returns a collection of EngineeringInvocationInfo objects describing the different actions of these objects. EngineeringInvocationInfo is described below.

Self-description Support on IEngineeringAssociation

Method Name	Return values
GetInvocationInfos()	Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object. EngineeringInvocationInfo is described below.

Self-description Support on IEngineeringComposition

Method Name	Return values
GetCreationInfos()	Returns a collection of EngineeringCreationInfo objects describing the different creation methods for items on this composition. EngineeringCreationInfo is described below.
GetInvocationInfos()	Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object. EngineeringInvocationInfo is described below.

Self-description Support on IEngineeringServiceProvider

Method Name	Return values
GetServiceInfos	Returns a collection of EngineeringServiceInfo objects describing the different services of these objects. EngineeringServiceInfo is described below.

Class EngineeringCompositionInfo

Attribute Name	Return values
Name	The name of the composition
Type	The type of the composition.

Class EngineeringAttributeInfo

Attribute Name	Return values
AccessMode	The level of access supported by the attribute. This attribute is combinable and is described in detail below.
Name	The name of the attribute.
CreateRelevance	The level to which the attribute is create relevant.
SupportedTypes	The potentially supported types of the attribute.

Class EngineeringInvocationInfo

Attribute Name	Return values
Name	The name of the action.
ParameterInfos	A collection of EngineeringInvocationParameterInfo objects describing any parameters that the action might require. EngineeringInvocationParameterInfo is described below.

Type	The return type of the action.

Class EngineeringServiceInfo

Attribute Name	Return values
Type	The type of the service as a System.Type object.

Enum EngineeringAttributeAccessMode

Enum Value	Return values
None	This is not a valid option.
Read	The attribute can be read.
ReadWrite	The attribute can be read and written
Write	The attribute can be written.

Enum EngineeringCreateRelevance

Enum Value	Return values
None	The attribute is not create relevant.
Mandatory	The attribute is required for the creation of an object.
Relevant	The attribute is optional for the creation of an object.

Class EngineeringInvocationParameterInfo

Attribute Name	Return values
Name	The name of the parameter.
Type	The type of the parameter as a System.Type object

Class EngineeringCreationInfo

Attribute Name	Return values
Type	The Type property returns a System.Type object
ParameterInfos	The ParameterInfos property returns a collection of EngineeringCreationParameterInfo objects

Class EngineeringCreationParameterInfo

Attribute Name	Return values
Name	The name of the parameter in question

Program code

AccessMode is a flags enum and its values can be combined like the following program code:



```
Copies the following program code to clipboard.

EngineeringAttributeAccessMode value =
EngineeringAttributeAccessMode.Read|EngineeringAttributeAccessMode.Write
```

Modify the following program code to find all attributes of an IEngineeringObject and to do changes on the access mode of those attributes.



```
Copies the following program code to clipboard.

...
IEngineeringObject engineeringObject = ...;
IList<EngineeringAttributeInfo> attributeInfos =
engineeringObject.GetAttributeInfos();
foreach(EngineeringAttributeInfo attributeInfo in
attributeInfos)
{
    switch (attributeInfo.AccessMode)
        {
        case EngineeringAttributeAccessMode.Read:
            ...
            break;
```

```
        case EngineeringAttributeAccessMode.Write:
            ...
            break;
        case
EngineeringAttributeAccessMode.Read|EngineeringAttributeAccessMode.Write
            ...
            break;
        }
}
...
```

Modify the following program code of how Create and GetCreationInfos are intended to be used together.



```
. . .
```

2.23 Setting attributes of Blocks, DBs & UDTs

Introduction

It is possible to write and read (general) attributes of the Blocks (written in any programming language), DBs and UDTs via the Openness API.

TechnologicalObjects are internally DBs, so this feature is also valid for TOs.

The same checking rules are to be applied on Name by using SetAttribute of the Openness API on block, DB (TO) or UDT which are applied in the GUI.

In case of rule violation an appropriate user exception will be thrown and the attribute value won't be changed.

This extension of the API doesn't affect the XML export / import.

General Attributes

Legend

- X: The attribute is relevant, write to xml, accessible with API
- -: The attribute is not accessible, the attribute does not exist

Attributes	Datatype	Availability via the API		
		CodeBlock	DB	UDT
AutoNumber	Boolean	X (xml); RW (API)	X (xml); RW (API)	-
CodeModifiedDate	DateTime	X (xml); R (API)	X (xml); R (API)	-
CompileDate	DateTime	X (xml); R (API)	X (xml); R (API)	-
CreationDate	DateTime	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
HeaderAuthor	String	X (xml); R (API)	X (xml); R (API)	-

HeaderFamily	String	X (xml); R (API)	X (xml); R (API)	-
HeaderName	String	X (xml); R (API)	X (xml); R (API)	-
HeaderVersion	System.Version	X (xml); R (API)	X (xml); R (API)	-
InstanceOfName	String	-	X (xml); R (API)	-
Interface	String	X (xml); - (API)	X (xml); - (API)	X (xml); - (API)
InterfaceModifiedDate	DateTime	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
IsConsistent	Boolean	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
IsKnowHowProtected	Boolean	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
MemoryLayout (sofar: Optimization)	MemoryLayout	X (xml); R (API)	X (xml); R (API)	-
ModifiedDate	DateTime	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
Name	String	X (xml); RW (API)	X (xml); RW (API)	X (xml); RW (API)
Number	Int32	X (xml); RW (API)	X (xml); RW (API)	-
ParameterModified	DateTime	X (xml); R (API)	X (xml); R (API)	-
ProgrammingLanguage	ProgrammingLanguage	X (xml); R (API)	X (xml); R (API)	-
SecondaryType	String	X (xml); R (API)	-	-

StructureModified	DateTime	X (xml); R (API)	X (xml); R (API)	-

2.24 Notes on performance of TIA Portal Openness

Root objects

You can specify several root objects in import files.

Example: You can create several text lists in an XML file instead of one text list for each XML file.

TIA Portal Openness functions

The first call of a TIA Portal Openness function can take longer than any subsequent calls of the TIA Portal Openness function.

Example: If you perform multiple configuration data exports one after the other, the first export can take longer than the subsequent exports.

Functions for projects and project data 3

3.1 Opening a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- The project to be opened is not open in any other instance of the TIA Portal.

Note

Undo of project upgrade

If you undo an upgrade of a project to V14SP1 after you have connected it to TIA Portal Openness, conflicts will occur.

Introduction

Use the `Projects.Open` method to open a project. Enter a path to the desired project in the `Projects.Open` method.

The `Projects.Open` method only accesses projects that were created with the current version of TIA Portal or which have been upgraded to the current version. If you access a project of a previous version with the `Projects.Open` method, an exception will be returned. Use the `OpenWithUpgrade` method, to open projects which have been made with previous versions of TIA Portal.

Program code

Modify the following program code to open a project:

```
Copies the following program code to clipboard.
private static Project ProjectOpen(string projectPath)
{
        try
        {
            // Refer Connecting to the TIA Portal section
            TiaPortal tiaPortal = new TiaPortal();
```

```
            Project project
=tiaPortal.Projects.Open(projectPath);
            if (project != null)
            {
            Console.WriteLine("Project Name: " +
project.Name);
            Console.WriteLine("Project Version: " +
project.Version);
            }
            return project
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Exception of type
{ex.GetType()} occurred: {ex.Message}");
            return null;
        }
}
```

Opening a UMAC protected project

You can also open a UMAC protected project. The overload of Open function takes an additional parameter of type UmacDelegate. This additional parameter allows the caller to specify a handler to be used during UMAC authentication. The new UmacDelegate is implemented with a method containing one parameter of type UmacCredentials. The UmacCredentials has two properties, 'Name' of type string, and 'Type' of type UmacUserType; and one method SetPassword with one parameter of type SecureString. The use of UmacUserType.Project indicates a UMAC scope of project whereas the use of UmacUserType.Global indicates a UMAC scope of application (i.e. controlled by a UMAC server).

Program code

```
Copies the following program code to clipboard.
public struct UmacCredentialsParameters
{
        public UmacUserType Type;
```

```
        public string Name;
        public SecureString Password;
}
private static void UMACProtectedProjectOpen(string
projectFilePath,
UmacCredentialsParameters umacParameters)
{
        try
     {
            void MyUmacDelegate(UmacCredentials
umacCredentials)
     {
         umacCredentials.Type = umacParameters.Type;
                    umacCredentials.Name =
umacParameters.Name;
         umacCredentials.SetPassword(umacParameters.Password);
     }
     // Refer Connecting to the TIA Portal section
     TiaPortal tiaPortal = new TiaPortal();

     Project project = tiaPortal.Projects.Open(new
FileInfo(projectFilePath), MyUmacDelegate);
     if (project != null)
     {
     try
             {
                    Console.WriteLine(project.Name);
Console.WriteLine(project.Version);
     }
               finally
{
project.Close();
     }
```

```
            }
        }
    catch(Exception ex)
    {
            Console.WriteLine(ex.ToString());
    }
}
```

Opening a UMAC protected project using new authentication

Before TIA Portal Openness V17, You can open the protected project by passing the user name and password through Umac delegate in Open Project API.

With TIA Portal Openness V17, UMAC have introduced new authentication mechanism by which you will able to open a protected project. The following new authentication mechanisms are Desktop Single Sign-On (SSO) and Anonymous User.

TIA Potal Openness V17 supports the following authentication mechanisms. .The Openness implementation is independent of settings in TIA Portal.

- Desktop Single Sign-On (SSO)
- Anonymous User
- Interactive log on
- Credential (Existing From TIA Portal V15.1)

In order to accommodate the additional authentication methods, Openness have introduced a new event name "Authentication" in TIA Portal object. The changes are introduced as a part of V17 Engineering. The Event "Authentication" when registered by the user in the Openness script, will be executed when open project is called. The Event "Authentication" will be executed only for Open Protected Project.



```
private static void OnAuthentication(object sender,
AuthenticationEventArgs e)
{
    // Refer Connecting to the TIA Portal section
    var tiaPortal = new TiaPortal();
    tiaPortal.Authentication += OnAuthentication;
    e.AuthenticationTypeProvider =
AuthenticationTypeProvider.Credentials;
}
```

You are be able to use the Additional Authentication Methods using the AuthenticationEventArgs event args. Using AuthenticationTypeProvider ,the mode of authentication is provided. From the above example , the crediantial mode is set.

Below are the values for AuthenticationTypeProvider.

- AuthenticationTypeProvider.DesktopSso : Open project with Single sign on user , no password asked
- AuthenticationTypeProvider.Anonymous : Open project with anonymous user , no password asked
- AuthenticationTypeProvider.Interactive : Open project with Interactive logon, User name and password to be given in user interface
- AuthenticationTypeProvider.Credentials : Open Project with User name and password given.

The event is applicable for opening the project in multiuser user environment.The above authentication methods is applicable when a protected project is opened in local session or server view.

If there are exceptions for example, a single sign-on session or a invalid user name / password scenarios are handled by EngineeringTargetInvocationException.

Opening multiple projects

You can open one primary project and multiple secondary projects at time in an instance of the TIA Portal. You can open a project as a primary or a secondary project. If a project is opened as primary, then this project is represented in the project navigation if the Openness application is attached to a TIA Portal and project is opened as secondary, then this project is not reflected in the user interface. Secondary projects are always opened as read-only. a user with read and write priviliges to UMAC projected project will have read-only priviliges when project is opened as secondary. A primary project does not need to be open in order to open a secondary project.

Any opened projects can be enumerated by using the ProjectComposition available on the TiaPortal instance. The order of the projects in the composition will be determined by the order in which the projects were opened. If a project is closed, the index of all projects will be recalculated.

Program code

```
Copies the following program code to clipboard.
private static void OpeningMultipleProjects()
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    Project project1 = tiaPortal.Projects.Open(new
FileInfo(@"D:\Project_1\Project_1.apXX"), null,
ProjectOpenMode.Primary);
    Project project3 = tiaPortal.Projects.Open(new
FileInfo(@"D:\Project_3\Project_3.apXX"), null,
ProjectOpenMode.Secondary);
    bool isPrimary = project3.IsPrimary;
}
```

Opening projects created with previous versions

Use the `OpenWithUpgrade` method to open a project which was created with the previous version of TIA Portal. The method will create a new, upgraded project and open it.

If you access a project created with an elder version than the previous, an exception will be returned.

Note

If you access a project created with the current version the project will just be opened.

Program code

Modify the following program code to open a project via the `OpenWithUpgrade` method:

```
Copies the following program code to clipboard.
private static void ProjectOpenWithUpgrade()
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
```

```
    Project project = tiaPortal.Projects.OpenWithUpgrade(new
FileInfo(@"D:\Some\Path\Here\Project.apXX"));
    if (project != null)
    {
        try
        {
         Console.WriteLine("Project Name: " + project.Name);
        }
        finally
        {
        project.Close();
        }
    }
}
```

Program code for UMAC protected project

You can also open a UMAC protected which has been created with a previous version of TIA Portal. An overload function of OpenWithUpgrade takes an additional parameter of type UmacDelegate. OpenWithUpgrade is also supported for secondary projects.

```
 Copies the following program code to clipboard.
private static void UMACProtectedProject(UmacDelegate
MyUmacDelegate)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    Siemens.Engineering.Project project =
tiaPortal.Projects.OpenWithUpgrade(new
    FileInfo(@"D:\Project_1\Project.apXX"), MyUmacDelegate);
}
```

3.2 Creating a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

Introduction

Projects can be created via TIA Portal Openness API

- by calling the Create method on ProjectComposition
- by calling the Create method on IEngineeringComposition

ProjectComposition.Create

Modify the following program code:

```
Copies the following program code to clipboard.
private static void ProjectCreate()
{
    TiaPortal tiaPortal = new TiaPortal();
    ProjectComposition projectComposition =
tiaPortal.Projects;
    DirectoryInfo targetDirectory = new
DirectoryInfo(@"D:\TiaProjects");
    // Create a project with name MyProject
    Project project =
projectComposition.Create(targetDirectory, "MyProject");
}
```

According to this example

- a folder "D:\TiaProjects\MyProject" will be created.
- a project file "D:\TiaProjects\MyProject\MyProject.aPXX" will be created.

Note

About parameter targetDirectory

The parameter targetDirectory can also represent an UNC (Universal Naming Convention) path, therefore a project can also be created on a network shared drive.

IEngineeringComposition.Create

Modify the following program code:

 Copies the following program code to clipboard.

```
public struct ProjectCreateParameters
{
   public DirectoryInfo TargetDirectory;
   public string Name;
   public string Author;
   public string Comment;
}
private static Project
ProjectAttributeCreate(ProjectCreateParameters parameters)
{
   try
   {
    TiaPortal tiaPortal = new TiaPortal();
    ProjectComposition projectComposition =
tiaPortal.Projects;
    // Prepare the create parameters
    var createParameters = new List<KeyValuePair<string,
object>>
    {
    new KeyValuePair<string, object>("TargetDirectory",
parameters.TargetDirectory), // Mandatory
    new KeyValuePair<string, object>("Name", parameters.Name)
// Mandatory
    };
    if (!string.IsNullOrEmpty(parameters.Author))
    {
    createParameters.Add(new KeyValuePair<string, object>
("Author", parameters.Author)); // Optional
    if (!string.IsNullOrEmpty(parameters .Comment))
    {
    createParameters.Add(new KeyValuePair<string, object>
("Comment", parameters.Comment)); // Optional
```

```
    }
    // Create a project with both mandatory and optional
parameters
    return ((IEngineeringComposition)projectComposition)
.Create(typeof(Project), createParameters) as Project;
    }
   catch (Exception ex)
   {
    Console.WriteLine($"Exception of type {ex.GetType()}
occurred: {ex.Message}");
    return null;
   }
}
```

According to this example:

- a folder "D:\TiaProjects\MyProject" will be created.
- a project file "D:\TiaProjects\MyProject\MyProject.aPXX" will be created with project attributes Author as "Bob" and Comment as "This project was created with openness".

Parameters for creating project with optional project attributes

Parameter	Data Type	Is Mandatory	Description
Author	String	No	Author of a project.
Comment	String	No	Comment for the project.
Name	String	Yes	Name of a project,
TargetDirectory	DirectoryInfo	Yes	Directory that will contain the created proeject folder.

3.3 Accessing general settings of the TIA Portal

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal

- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness to access general settings of the TIA Portal:

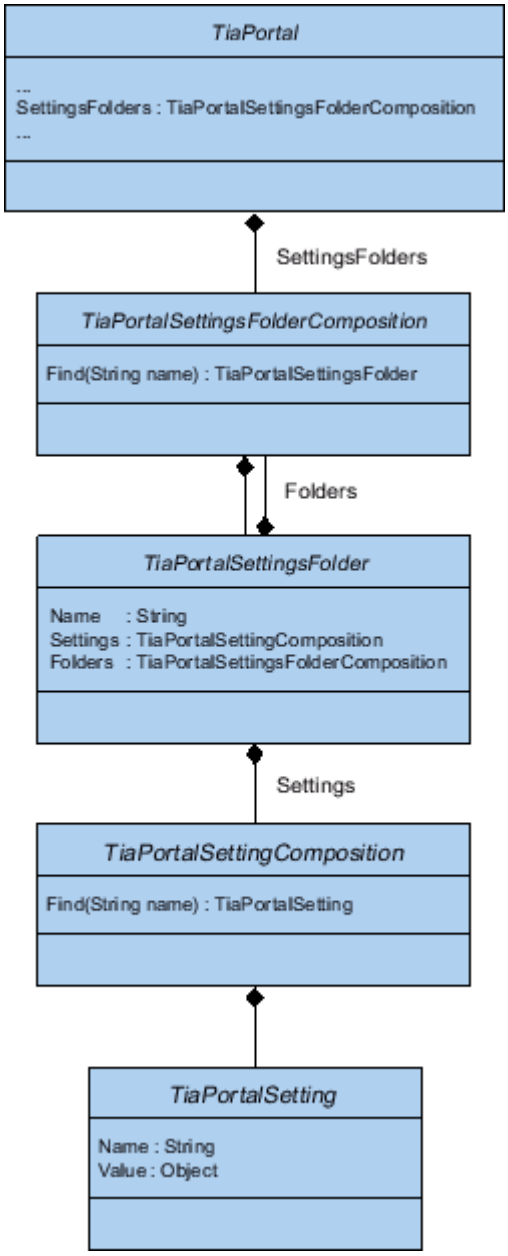
- Current user interface language
- "Search in project" option to create the search index needed for searching within a project.

The following table shows the details of the accessible settings in the "General" section of the TIA Portal settings. The `TiaPortalSettingsFolder` instance will have the name "General".

Setting name	Data type	Writeable	Description
"SearchInProject"	System.Boolean	r/w	Enables or disables the creation of the search index needed for searching within a project.
"UserInterfaceLanguage"	System.CultureInfo	r/w	Indicates the active user interface language of the TIA Portal or the specification of the active user interface language.

The access to these settings is provided via the `TiaPortalSettingsFolder` class. The `TiaPortalSettingsFolder` class will be accessible via the `Settings` attribute on the `TiaPortal` class.

The following figure shows the specific settings in TIA Portal Openness:



Program code: Search in project

Modify the following program code to activate/deactivate the "Search in project" option.

```
Copies the following program code to clipboard.
private static void SetSearchInPoject()
{
    TiaPortal tiaPortal = new TiaPortal();
    TiaPortalSettingsFolder generalSettingsFolder =
    tiaPortal.SettingsFolders.Find("General");
    TiaPortalSetting searchSetting =
```

```
    generalSettingsFolder.Settings.Find("SearchInProject");
        if (((bool)searchSetting.Value))
        {
            searchSetting.Value = false;
        }
   }
```

Program code: User interface language

Modify the following program code to access the current user interface language.

```
Copies the following program code to clipboard.
private static void SetUILanguage()
{
    TiaPortal tiaPortal = new TiaPortal();
    TiaPortalSettingsFolder generalSettingsFolder =
    tiaPortal.SettingsFolders.Find("General");
    TiaPortalSetting UILanguageSetting =
generalSettingsFolder.
    Settings.Find("UserInterfaceLanguage");
        if (((CultureInfo)UILanguageSetting.Value) !=
        CultureInfo.GetCultureInfo("de-DE"))
        {
            UILanguageSetting .Value =
CultureInfo.GetCultureInfo("de-DE");
        }

}
```

See also

Opening a project

3.4 Archiving and Retrieving a project

Requirement

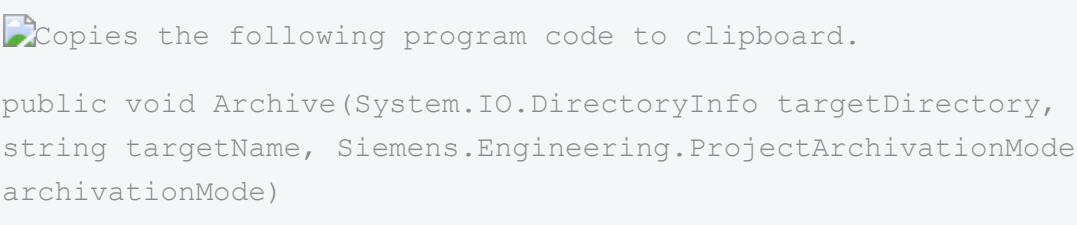
- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project
- A project is save
  See Saving a Project

Introduction

You can use the TIA Portal Openness to archive the opened and saved project state prior to further any changes and then later you can retrieve the archived project.

Archiving a project

You can archive a TIA Portal project using TIA Openness API available on Siemens.Engineering.Project object.



'targetName' is the name of the file created may it be archived or non archived. This file may or may not contain any file extensions. For ProjectArchivationMode value as Compressed and DiscardRestorableDataAndCompressed, the archived file name is as it is provided by you. If you don't provide any extension or extension apart from "zap15_1" or "zap14" etc., then this archived project could not be retrieved from TIA Portal outside the Openness API. If you don't not choose Compressed or DiscardRestorableDataAndCompressed the result will be the default file structure of a project of the current version.

Note

You must have saved the project before calling Archiving API. In case the project contains any unsaved changes, archive would throw an EngineeringTargetInvocationException.

Project ArchivationMode

The **ProjectArchivationMode** enumeration have four values.

ProjectArchivationMode	Description
None	• No special action are taken with the orginial files. Mode is similiar to a "save as" operation. • No compressed zip file is created in this mode. • The difference with SaveAs in this case is that the Archive will not change the persistence location to the new Archived folder whereas SaveAs does that
DiscardRestorableData	• The file storage stores the project in an internal data file and this file grows whenever a project data modification happens. In the case of DiscardRestorableData mode this data file is reorganized (only latest version of the objects are stored and history is removed from the file) and Intermediate data, the files of the IM directory and tmp directory are not copied to the archive location. • No compressed zip file is created in this mode.
Compressed	• The tmp project folder structure, created by Archiving, is compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.

DiscardRestorableDataAndCompressed	• In the tmp project folder structure, created by Archiving, the restorable data is discarded and then compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.

Program code: Archiving a poject

Modify the following program code to access the archive project without an extension:



```
Copies the following program code to clipboard.
public void ArchivingProjectWithoutExtension()
{
    var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);
    var projectFilePath = @"E:\Sample1\Sample1.ap15_1";
    var project = tiaPortal.Projects.Open(new
FileInfo(projectFilePath));
    var archiveDirectory = @"E:\Archive";
   project.Archive(new DirectoryInfo(archiveDirectory),
   "ArchiveProjectNameWithoutExtension",
ProjectArchivationMode.Compressed);
}
```

Modify the following program code to access the archive project with an extension as '.archive':



```
Copies the following program code to clipboard.
public void ArchivingProjectWithExtension()
{
    var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);
    var projectFilePath = @"E:\Sample1\Sample1.ap15_1";
    var project = tiaPortal.Projects.Open(new
FileInfo(projectFilePath));
    var archiveDirectory = @"E:\Archive";
    project.Archive(new DirectoryInfo (archiveDirectory),
```

```
    "ArchiveProjectName.archive",
ProjectArchivationMode.Compressed);
}
```

Note

You are free to use any extension. The behavior of Archive will be same with/without extension.

Retrieving a project

You can use the Openness API to retrieve an archived TIA Portal project. You can only retrieve a compressed archive. For Archived project with 'ProjectArchivationMode.None' or 'ProjectArchivationMode.DiscardRestorableData' enumeration value cannot be retrieved by this API. This API is available on the "Siemens.Engineering.ProjectComposition" object.

```
Copies the following program code to clipboard.

public Siemens.Engineering.Project
Retrieve(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory)
```

For retrieving a UMAC protected project an overloaded API with the UmacDelegate is provided for getting the credentials, then the following API definition is used

```
Copies the following program code to clipboard.

public Siemens.Engineering.Project
Retrieve(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory,
Siemens.Engineering.UmacDelegate umacDelegate)
```

Retrieve also supports ProjectOpenMode that can be specified as "Primary" or "Secondary", then the following API definition is used

```
Copies the following program code to clipboard.

public Siemens.Engineering.Project
```

```
Retrieve(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo
targetDirectory,Siemens.Engineering.UmacDelegate
umacDelegate, Siemens.Engineering.ProjectOpenMode
projectOpenMode)
```

Note

umacDelegate could be passed as null if the project is not protected

If the archived project is of a previous TIA Portal version then you have to call the RetrieveWithUpgrade API, then the following API definition is used



```
public Siemens.Engineering.Project
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory);
```

If the archived project is of a previous TIA Portal version and UMAC protected then another overloaded API with UmacDelegate is available, then the following API definition is used.



```
public Siemens.Engineering.Project
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory,
Siemens.Engineering.UmacDelegate umacDelegate)
```

If the archived project is of a previous TIA portal version and project open mode also needs to be specified, then the following API definition is used.



```
public Siemens.Engineering.Project
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo
targetDirectory,Siemens.Engineering.UmacDelegate
umacDelegate, Siemens.Engineering.ProjectOpenMode
projectOpenMode)
```

Note

umacDelegate could be passed as null if the project is not protected. If the project is protected then the user credentials of an admin user is required to perform the

RetrieveWithUpgrade action

Program code: Retrieving a project

Modify the following program code to access the retrieve project:

```
Copies the following program code to clipboard.
public void RetrieveProject()
{
    var archivePath = @"E:\Archive\Sample1.zap15_1";
    var retrievedProjectsDirectory = @"E:\RetrievedProjects";
    var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);
    tiaPortal.Projects.Retrieve(new FileInfo(archivePath),
    new DirectoryInfo(retrievedProjectsDirectory));
}
```

See also

Connecting to the TIA Portal
Opening a project
Saving a project

3.5 Accessing read-only TIA Portal project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

Using TIA Portal Openness, you can perform select operations while working with a read-only TIA Portal project. You can have access to read-only project, but you will not be able to use full set of features that are available to a user with read-write access. For example, a user with read-only credentials can use Openness to open a UMAC protected project as described in Opening a project. This functionality does not include Reference projects.

The list of Openness features that are available to you while accessing a read-only project can be categorized into two sets of features - Inherent and Enabled non-modifying actions:

Inherent functionality

- GetAttribute(s) or using the getter for any attribute on any accessible object
- GetComposition on any accessible object
- GetService on any accessible object
- Find actions on any accessible object
- Navigation on any accessible object
- Determining the existence of accessible objects and accessing those objects in compositions and associations
- System.Object methods on any accessible object

Enabled non-modifying actions

- Project.Close (...)
- PlcBlock.ShowInEditor ()
- CaxProvider.Export (Device,...)
- CaxProvider.Export (Project,...)

See also

Opening a project

3.6 Accessing languages

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

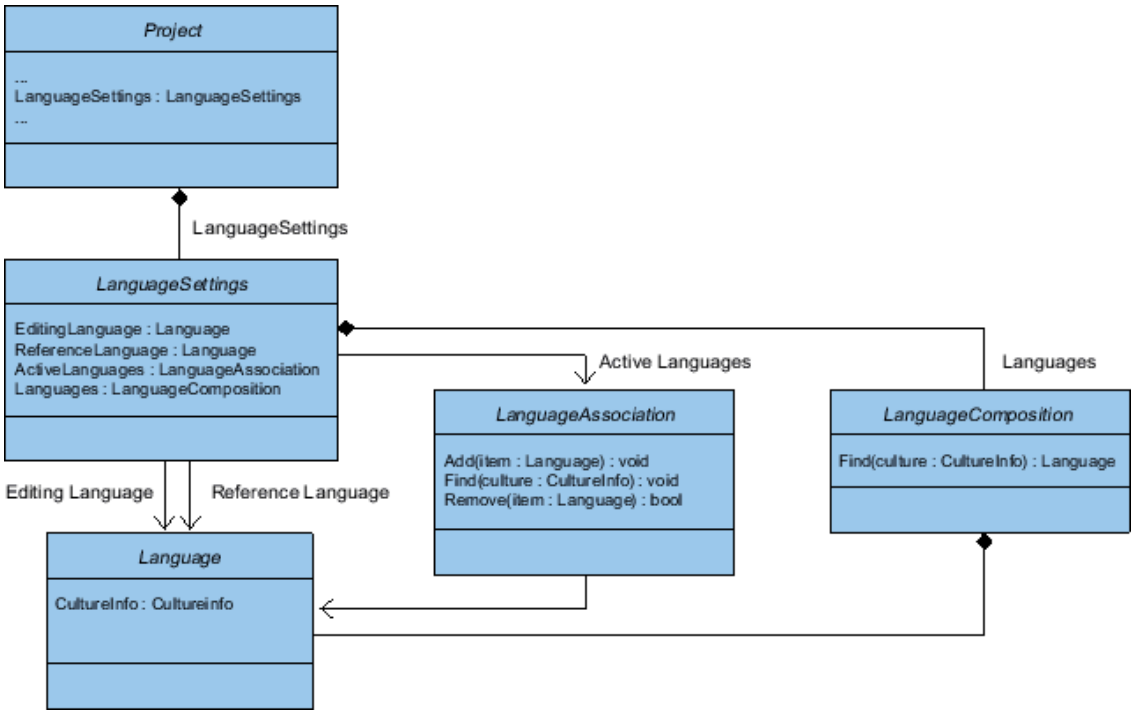
In the TIA Portal you can set and manage the project language in the Editor "Project languages".

TIA Portal Openness supports the following access to the project languages:

- Iterating through supported languages.
- Searching through the collection of supported languages by `System.Globalization.CultureInfo`.
- Accessing individual languages. Each Language object will contain a single read-only attribute `Culture` of type `System.Globalization.CultureInfo.`
- Accessing a collection of active languages.

- Searchring through the collection of active languages by `System.Globalization.CultureInfo`.
- Adding a language to a collection of active languages.
- Removing a language from a collection of active languages.
- Setting an editing language.
- Setting a reference language.

The functionalities are provided by the `LanguageSettings` object. The following figure shows model, which is provided by TIA Portal Openness:



Program code: Setting languages

Modify the following program code to set a language. If you set an inactive language via TIA Portal Openness, the language will be added to the active languages collection.

```
Copies the following program code to clipboard.
private static void SettingLanguages()
{
   TiaPortal tiaPortal = new TiaPortal();
   Project project = tiaPortal.Projects.
   Open(new FileInfo(@"D:\Project_3\Project_2.apXX"));
   LanguageSettings languageSettings =
project.LanguageSettings;
```

```
   LanguageComposition supportedLanguages =
languageSettings.Languages;
   LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;
   Language supportedGermanLanguage = supportedLanguages.
   Find(CultureInfo.GetCultureInfo("de-DE"));
   activeLanguages.Add(supportedGermanLanguage);
   languageSettings.EditingLanguage =
supportedGermanLanguage;
   languageSettings.ReferenceLanguage =
supportedGermanLanguage;
}
```

Program code: Deactivating an active language

Modify the following program code to deactivate an active language. If you deactivate a language which is used as reference or editing language the language selected will be consistent with the behavior in the user interface.



```
Copies the following program code to clipboard.
private static LanguageSettings
DeactivatingLanguages(TiaPortal tiaPortal, string
projectPath)
{
   try
   {
   Project project = tiaPortal.Projects.Open(new
FileInfo(projectPath));
   LanguageSettings languageSettings =
project.LanguageSettings;
   LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;
   Language activeGermanLanguage = activeLanguages.
   Find(CultureInfo.GetCultureInfo("de-DE"));
   activeLanguages.Remove(activeGermanLanguage);
   return languageSettings;
```

```
    }
    catch(Exception ex)
    {
    Console.WriteLine($"Exception of type {ex.GetType()}
occurred: {ex.Message}");
    return null;
    }
}
```

See also

Hierarchy of hardware objects of the object model

3.7 Determining the object structure and attributes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can determine the navigation structure through the object hierarchy with the `IEngineeringObject` interface. The result is returned as a list:

- Child objects
- Child compositions
- All attributes

Signature

Use the `GetAttributeInfos` method to determine attributes.

```
IList<EngineeringAttributeInfo> IEngineeringObject.GetAttributeInfos();
```

Program code: Determining objects or compositions

Use the following program code to display all composition names:

```
Copies the following program code to clipboard.
```

```
private static void
DisplayCompositionInfos(IEngineeringObject obj)
{
    IList<EngineeringCompositionInfo> compositionInfos =
obj.GetCompositionInfos();
    foreach (EngineeringCompositionInfo compositionInfo in
compositionInfos)
    {
    Console.WriteLine(compositionInfo.Name);
    }
}
```

Modify the following program code if you know the return value:



```
Copies the following program code to clipboard.
private static DeviceItemComposition
GetDeviceItemComposition(Device device)
{
    IEngineeringCompositionOrObject composition =
((IEngineeringObject) device).GetComposition("DeviceItems");
    DeviceItemComposition deviceItemComposition =
(DeviceItemComposition)composition;
    return deviceItemComposition;
}
```

Program code: Determining attributes

Modify the following program code to return attributes of an object with specific access rights in a list:



```
Copies the following program code to clipboard.
private static string
DisplayAttributenInfos(IEngineeringObject obj)
{
    IList<EngineeringAttributeInfo> attributeInfos =
obj.GetAttributeInfos();
```

```
    foreach (EngineeringAttributeInfo attributeInfo in
attributeInfos)
    {
    Console.WriteLine("Attribute: {0} - AccessMode {1} ",
attributeInfo.Name,
    attributeInfo.AccessMode);
    switch (attributeInfo.AccessMode)
    {
    case EngineeringAttributeAccessMode.Read:
Console.WriteLine
    ("Attribute: {0} Read Access", attributeInfo.Name);
    break;
    case EngineeringAttributeAccessMode.Write:
Console.WriteLine
    ("Attribute: {0} Write Access", attributeInfo.Name);
    break;
    case EngineeringAttributeAccessMode.Read |
EngineeringAttributeAccessMode.Write:
    Console.WriteLine("Attribute: {0} - Read and Write
Access",attributeInfo.Name);
    break;
    }
    }
    Object nameAttribute = obj.GetAttribute("Name");
    string name = nameAttribute?.ToString();
    return name;
}
```

3.8 Access software target

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

See Opening a project

Program code

Modify the following programm code to make a software target available:



```
Copies the following program code to clipboard.

private static void AccessSoftwareTarget(DeviceItem
deviceItem)
{
    SoftwareContainer softwareContainer =
((IEngineeringServiceProvider)deviceItem).
    GetService<SoftwareContainer>();
    if (softwareContainer != null)
    {
    Software software = softwareContainer.Software;
    }
}
```

Modify the following programm code to access the software attributes:



```
Copies the following program code to clipboard.

private static void AccessSoftwareAttribute(DeviceItem
deviceItem)
{
    SoftwareContainer softwareContainer =
((IEngineeringServiceProvider)deviceItem).
    GetService<SoftwareContainer>();
    if (softwareContainer != null)
    {
    PlcSoftware software = softwareContainer.Software as
PlcSoftware;
    string name = software.Name;
    }
}
```

3.9 Accessing and enumerating multilingual texts

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

Multilingual texts in the TIA Portal are e. g. Project.Comment, PlcTag.Comment etc. In TIA Portal Openness, the multilingual texts are represented by the `MultilingualText` object. A `MultilingualText` object is composed of `MultilingualTextItemComposition`.

`MultilingualTextItemComposition` supports the following `Find` method:

- `Find(<language: Siemens.Engineering.Language>):MultilingualTextItem`

Each `MultilingualTextItem` provides the following attributes:

Attribute name	Data type	Writable	Description
Language	Siemens.Engineering.Language	r/o	Language of this item.
Text	System.String	r/w	Text provided for this language.

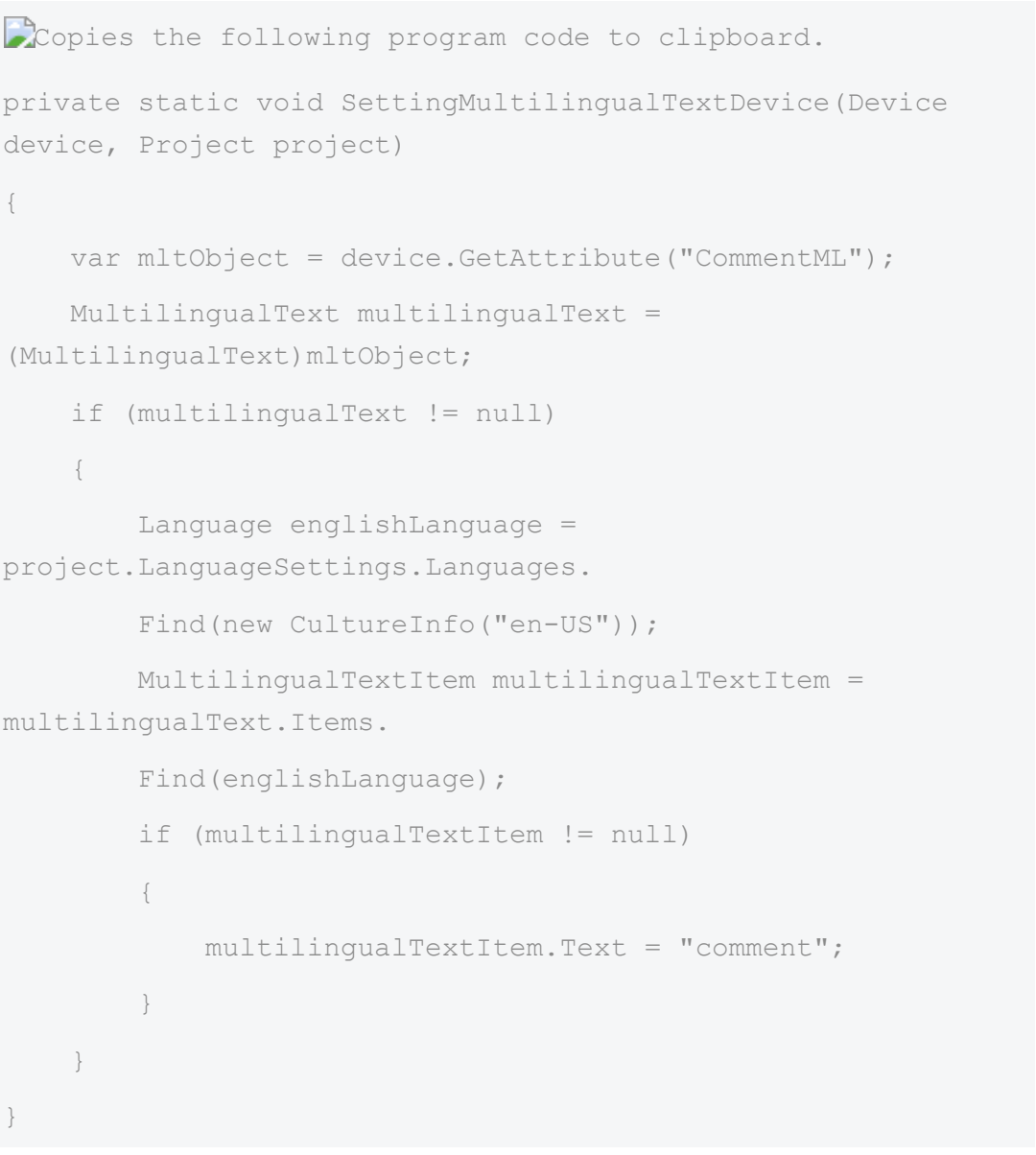
Program code: Set multilingual text

```
Copies the following program code to clipboard.
private static void SettingMultilingualText(Project project)
{
    Language englishLanguage =
project.LanguageSettings.Languages.
    Find(new CultureInfo("en-US"));
    MultilingualText comment = project.Comment;
    MultilingualTextItemComposition mltItemComposition =
comment.Items;
    MultilingualTextItem englishComment =
mltItemComposition.Find(englishLanguage);
```

```
    englishComment.Text = "English comment";
}
```

Program code: Set multilingual text for devices

Modify the following program code to set the multilingual text for devices and device items:



3.10 Updating project properties

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to update the simulation support and virtual PLC support property of a project in order to create a plc program. The setting for simulation support and virtual PLC support property is available for virtual SINUMERIK controller and SIMATIC PLCs simulated with PLCSIM Advanced .

The following attribute is supported to access the "simulation support" and "Virtual PLC support" property of a TIA Portal project. The API is available on the "Siemens.Engineering.Project" object.

Attribute name	Data type	Access	Description
IsSimulationDuringBlockCompilationEnabled	System.Boolean	Read / Write	To indicate whether support for simulation during block compilation is enabled for the project
IsVirtualPlcDuringBlockCompilationEnabled	System.Boolean	Read / Write	To indicate whether support for virtual plc during block compilation is enabled for project

Program code

Modify the following program code to access the attribute value:

```
Copies the following program code to clipboard.
private static void UpdateProjectProperty(Project project)
{
    // Read the attribute value
    var attributeValue =
project.IsSimulationDuringBlockCompilationEnabled;
```

```
    var attributeValueForVirtualPLC =
project.IsVirtualPlcDuringBlockCompilationEnabled;
    //Write the attribute value to false
    project.IsSimulationDuringBlockCompilationEnabled =
false;
    //Write the attribute value to True
    project.IsVirtualPlcDuringBlockCompilationEnabled = true;
}
```

See also

Opening a project

3.11 Read project related attributes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Introduction

By using this function you can get project related attributes from the TIA Portal Openness API. The provided information contains project attributes, project history and products utilized by the project.

Project attributes

The project attributes provide the following information:

Attribute name	Data type	Writeable	Description
Author	System.String	r/o	The author of the project
Comment	Siemens.Engineering.MultilingualText	r/o	The comment of the project

Copyright	System.String	r/o	The copyright statement of the project
CreationTime	System.DateTime	r/o	The time when project has been created
Family	System.String	r/o	The family of the project
IsModified	System.Boolean	r/o	Returns true if the project has been modified
LanguageSettings	Siemens.Engineering.LanguageSettings	r/o	Handles proct languages
LastModified	System.DateTime	r/o	The time when project was last modified
LastModifiedBy	System.String	r/o	Who made the last modification
Name	System.String	r/o	The name of the project
Path	System.IO.FileInfo	r/o	The absolute path of the project
Size	System.Int64	r/o	The size of the project in KB
Version	System.String	r/o	The version of the project

Modify the following program code to access project related attributes:

```
Copies the following program code to clipboard.
private static void AccessProjectAttributes(Project project)
{
    string author = project.Author;
```

```
    string name = project.Name;
    DateTime creationTime = project.CreationTime;
    DateTime modificationTime = project.LastModified;
    string lastModifiedBy = project.LastModifiedBy;
    string version = project.Version;
    MultilingualText comment = project.Comment;
    string copyright = project.Copyright;
    string family = project.Family;
    Int64 size = project.Size;
    LanguageSettings languageSettings =
project.LanguageSettings;
}
```

Modify the following programm code to enumerate the project languages:

```
Copies the following program code to clipboard.
private static void AccessProjectLanguage(Project project)
{
   LanguageComposition languages =
project.LanguageSettings.Languages;
   foreach (Language language in languages)
   {
   CultureInfo lang = language.Culture;
   }
}
```

Modify the following program code to get comment text:

```
Copies the following program code to clipboard.
private static void RetrieveCommentText(Project project)
{
    Language english =
project.LanguageSettings.ActiveLanguages.
    Find(CultureInfo.GetCultureInfo("en-US"));
    MultilingualText projectComment = project.Comment;
```

```
    MultilingualTextItem textItem =
project.Comment.Items.Find(english);
    string text = textItem.Text;
}
```

Project history

The project history is a composition of `HistoryEntry` objects, which contain the following information:

Attribute name	Data type	Writeable	Description
Text	System.String	r/o	The event description
DateTime	System.DateTime	r/o	The time when the event was occured

Modify the following program code to enumerate through HistoryEntries and access their attributes:

```
 Copies the following program code to clipboard.
private static void AccessHistoryEntryAttributes(Project
project)
{
    HistoryEntryComposition historyEntryComposition =
project.HistoryEntries;
    foreach (HistoryEntry historyEntry in
historyEntryComposition)
    {
    string entryText = historyEntry.Text;
    DateTime entryTime = historyEntry.DateTime;
    }
}
```

Note

The text attribute of `HistoryEntry` contains a string in the same language as UI. If a TIA Portal Openness application is attached to a TIA Portal with no UI, the string

is always in English

Used Products

The object `UsedProduct` includes the following information:

Attribute name	Data type	Writeable	Description
Name	System.String	r/o	The name of the product used
Version	System.String	r/o	The version of the product

Modify the following program code to enumerate through `UsedProduct` and access the attributes.



3.12 Deleting project graphics

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Program code

Modify the following program code to delete a project graphics:

```
Copies the following program code to clipboard.
//Deletes a single project graphic entry
public static void DeletesSingleProjectGraphicEntry(Project project)
{
    MultiLingualGraphicComposition graphicsAggregation = project.Graphics;
    MultiLingualGraphic graphic = graphicsAggregation.Find("Graphic XYZ");
    graphic.Delete();
}
```

3.13 Compiling a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- All devices are "Offline".

Application

The API interface supports the compilation of devices and program blocks. The compilation result is returned as an object. Depending on type of the object HW or SW or HW/SW compilation will be provided. The following object types are supported:

- `Device - HW & SW`
- `Device` with failsafe CPU - `SW` with switched-off F-activation property
- `DeviceItem - HW`
- `CodeBlock - SW`
- `DataBlock - SW`
- `HmiTarget - SW`
- `PlcSoftware - SW`

- `PlcType - SW`
- `PlcBlockSystemGroup - SW`
- `PlcBlockUserGroup - SW`
- `PlcTypeSystemGroup - SW`
- `PlcTypeUserGroup - SW`

Note

Time stamp format

All time stamps are in UTC. If you want to see the local time you can use DateTime.ToLocalTime().

Note

If a password has been assigned to the safety program, you are required to a log on to the safety program to compile software changes. Otherwise, the process is canceled and an exception is triggered.

Signature

Use the `ICompilable` method for compilation.

```
ICompilable compileService =
iEngineeringServiceProvider.GetService<ICompilable>();
CompilerResult result = compileService.Compile();
```

Note

All devices must be "Offline" before you start compiling.

Program code

Modify the following program code to compile the software changes of an object of the type `HmiTarget`:

```
Copies the following program code to clipboard.
public static void CompileHmiTarget(HmiTarget hmiTarget)
{
    ICompilable compileService =
hmiTarget.GetService<ICompilable>();
```

```
    CompilerResult result = compileService.Compile();
}
```

Modify the following program code to compile the software changes of an object of the type `PlcSoftware`:

```
public static void CompilePlcSoftware(PlcSoftware
plcSoftware)
{
    ICompilable compileService =
plcSoftware.GetService<ICompilable>();
    CompilerResult result = compileService.Compile();
}
```

Modify the following program code to compile the software changes of an object of the type `CodeBlock`:

```
public static void CompileCodeBlock(PlcSoftware plcSoftware)
{
    CodeBlock block =
plcSoftware.BlockGroup.Blocks.Find("MyCodeBlock") as
CodeBlock;
    if (block != null)
    {
    ICompilable compileService =
block.GetService<ICompilable>();
    CompilerResult result = compileService.Compile();
    }
}
```

Modify the following program code to evaluate the compilation result:

```
private void WriteCompilerResults(CompilerResult result)
{
    Console.WriteLine("State:" + result.State);
```

```
    Console.WriteLine("Warning Count:" +
result.WarningCount);
    Console.WriteLine("Error Count:" + result.ErrorCount);
    RecursivelyWriteMessages(result.Messages);
}
private void
RecursivelyWriteMessages(CompilerResultMessageComposition
messages, string indent = "")
{
    indent += "\t";
    foreach (CompilerResultMessage message in messages)
    {
    Console.WriteLine(indent + "Path: " + message.Path);
    Console.WriteLine(indent + "DateTime: " +
message.DateTime);
    Console.WriteLine(indent + "State: " + message.State);
    Console.WriteLine(indent + "Description: " +
message.Description);
    Console.WriteLine(indent + "Warning Count: " +
message.WarningCount);
    Console.WriteLine(indent + "Error Count: " +
message.ErrorCount);
    RecursivelyWriteMessages(message.Messages, indent);
    }
}
```

3.14 Modifying project in UMAC

Introduction

You can use the TIA Portal Openness to modify project in user management and access right. With this Function Right , When a project is protected, "Modify Project via Openness API" can be assigned to a role which in turn can be assigned to user.

There are certain behavior changes in TIA Portal Openness when you are having / without having this function right in a protected project and affected when a protect project is opened with a particular user credentials. The changes for Openness API is applicable only on protected

project and on project which is opened with ProjectOpenMode as primary.The ProjectOpenMode as secondary is not considered.

Changes in behaviors with function right

- When you are having "Modify project via Openness API" function right along with Project Read Write rights , you are given with full access to Openness API i.e you can perform any operation using Openness API.
- When you are not having "Modify project via Openness API" function right along with Project Read Write rights / Project Read rights, you are entitled to Read only operations in Openness API. The examples of read only operations are export, navigation to a device etc. In this scenario, if you want to do an operation like import which is a data oriented operation (Change of project data in TIA portal) , then you would get an exception depending on the function rights assigned to you.
- When there are user interface related Openness API's example (ShowInEditor), the user interface related function rights does not depend on "Modify project via Openness API" , it depends on GUI function right which should be handled by the respective GUI function right. Below API's are the exceptions from point 1 and 2 . Openness API will bypass the Function Right "Modify project via Openness API" and the API's are allowed irrespective of Function Right "Modify project via Openness API" set / not set.
  - project.ShowHwEditor(View.Network)
  - project.ShowHwEditor(View.Topology)
  - device.ShowInEditor(View.Device)
  - plcBlock.ShowInEditor()
  - plcType.ShowInEditor()
  - plcTagTable.ShowInEditor()
  - plcForceTable.ShowInEditor()
  - plcWatchTable.ShowInEditor()

3.15 Saving a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

To save a project

- Use the `Save()` method to save a project

- Use the `SaveAs()` method to save a project with a different name or in a different directory

Program code

Modify the following program code open and save a project:



```
Copies the following program code to clipboard.

private static void SaveProject(TiaPortal tiaPortal, Project
project)
{
   //Use the code in the try block to open and save a project
   try
   {
   //please adapt the path and the extension apx
   //to the installed version of TIA Portal
   project = tiaPortal.Projects.Open(new
FileInfo(@"Some\Path\MyProject.apx"));
   //begin of code for further implementation
   //...
   //end of code'
   project.Save();
   }
   //Use the code in the final block to close a project
   finally
   {
   if (project != null)
   project.Close();
   }
}
```

Modify the following program code save a project with a different name or in a different location:



```
Copies the following program code to clipboard.

private static void SaveProjectWithDiffName()
```

```
{
    TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);
    FileInfo fileInfoExistingProject = new
FileInfo(@"D:\SampleProjects\SampleProject.apXX");
    DirectoryInfo dirInfoSaveAsProject = new
DirectoryInfo(@"D:\SampleProjects\SampleProjectSaveAs");
    Project sampleProject =
portal.Projects.Open(fileInfoExistingProject );
    sampleProject.SaveAs(dirInfoSaveAsProject);
}
```

3.16 Closing a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Program code

Modify the following program code to close a project:

```
Copies the following program code to clipboard.
public static void CloseProject(Project project)
{
project.Close();
}
```

3.17 Export/Import system diagnostics settings

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  SeeConnecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to export and import system diagnostics settings for a TIA Portal project. The format for export and import is .dat file.

The following service and class are used to export and import the setting for system diagnostics.

Name	Description
SystemdiagnosticsSettingsDataProvider	Service class to provide a method to export and import
SystemdiagnosticsSettingsExportImportResults	To return the result of Export or Import operation

The SystemdiagnosticsSettingsExportImportResultState ENUM has following values:

ENUM	Description	Value
SystemdiagnosticsSettingsExportImportResults	Returns the result whether the file is saved/settings are changed successfully or not.	Error
		Success
		Warning

Program code: Export system diagnostics settings

Modify the following program code to export settings for system diagnostics:



```
public static void ExportSystemDiagnosticSettings(Project
project)
{
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsDataProvider settingsProvider =
project.
    GetService<Siemens.Engineering.HW.Systemdiagnostics.
    Settings.SystemdiagnosticsSettingsDataProvider>();
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsExportImportResult exportResult
= settingsProvider.
```

```
    Export(new FileInfo(@"D:\Temp\SysDiagSettings.dat"));
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsExportImportResultState
resultState = exportResult.State;
}
```

Program code: Import system diagnostics settings

Modify the following program code to import settings for system diagnostics:

```
Copies the following program code to clipboard.
public static void ImportSystemDignosticSettings(Project
project)
{
   Siemens.Engineering.HW.Systemdiagnostics.Settings.
   SystemdiagnosticsSettingsDataProvider settingsProvider =
   project.GetService<Siemens.Engineering.HW.Systemdiagnostics.
   Settings.SystemdiagnosticsSettingsDataProvider>();
   Siemens.Engineering.HW.Systemdiagnostics.Settings.
   SystemdiagnosticsSettingsExportImportResult importResult =
   settingsProvider.Import(new
FileInfo(@"D:\Temp\SysDiagSettings.dat"));
   Siemens.Engineering.HW.Systemdiagnostics.Settings.
   SystemdiagnosticsSettingsExportImportResultState resultState
= importResult.State;
}
```

3.18 Identifying cross session object for Hardware & Software

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness application to uniquely identify enginnering objects.

To uniquely identify the engineering objects, you can use the ObjectIdentifierProvider service defined under the Siemens.Engineering namespace.

The TIA Portal Openness supports the following object types for unique identification:

- Siemens.Engineering.HW.Device
- Siemens.Engineering.HW.DeviceItem
- Siemens.Engineering.SW.Blocks.CodeBlock
- Siemens.Engineering.SW.Blocks.DataBlock
- Siemens.Engineering.SW.Blocks.FB
- Siemens.Engineering.SW.Blocks.FC
- Siemens.Engineering.SW.Blocks.OB
- Siemens.Engineering.SW.Blocks.GlobalDB
- Siemens.Engineering.SW.Blocks.InstanceDB
- Siemens.Engineering.SW.Blocks.ArrayDB
- Siemens.Engineering.SW.Tags;
- Siemens.Engineering.SW.Units;
- Siemens.Engineering.SW.TechnologicalObjects.TechnologicalInstanceDB
- Siemens.Engineering.SW.Types.PlcStruct

Method

The ObjectIdentifierProvider service provides the following method details through the TIA Portal Openness:

Method name	Data type	Description
GetIdentifier()	string	To retrieve unique id for particular engineering object
Find()	IEngineeringObject	To retrieve back engineering object from unique id.

Exception

You can encounter the exception in the following cases:

- If you have invoked GetIdentifier method with null as argument.

- If you have invoked GetIdentifier method with disposed object
- If you have invoked Find method with null as argument.

Program code

To access an engineering object from unique id modify the following program code:



```
Copies the following program code to clipboard.

private static void IdentifyCrossSectionObject(TiaPortal
tiaPortal)
{
    Project project = tiaPortal.Projects.Open(projectPath);
    ObjectIdentifierProvider objectIdentifierProvider =
project.GetService<ObjectIdentifierProvider>();
    //Get the ObjectIdentifierProvider service
    Device device = project.Devices.First();
    string deviceUniqueId =
objectIdentifierProvider.GetIdentifier(device);
    //Fetch unique id for device object
    Device device =
objectIdentifierProvider.Find(deviceUniqueId);
    //Find device object using unique id
    DeviceItem deviceItem =
device.DeviceItems.FirstOrDefault();
    string deviceItemUniqueId =
objectIdentifierProvider.GetIdentifier(deviceItem);
    //Fetch unique id for deviceItem object
    DeviceItem deviceItem =
objectIdentifierProvider.Find(deviceItemUniqueId);
    //Find deviceItem object using unique id
}
```

To access unique Identify engineering objects with exception handling modify the following program code:



```
Copies the following program code to clipboard.
```

```
private static void IdentifyCrossSectionObject(TiaPortal
tiaPortal)
{
     Project project = tiaPortal.Projects.Open(projectPath);
     ObjectIdentifierProvider objectIdentifierProvider =
project.
     GetService<ObjectIdentifierProvider>();
     try
     {
      objectIdentifierProvider.GetIdentifier(null);
      //Invoke GetIdentifier null
     }
     catch(EngineeringTargetInvocationException)
     {
      // new use case and new behavior.
     }
     Device device = project.Devices.First();
     device.Delete();
     try
     {
      objectIdentifierProvider.GetIdentifier(device);
      //Invoke GetIdentifier with disposed object
     }
     catch(EngineeringObjectDisposedException)
     {
      // new use case and new behavior.
     }
     try
     {
       objectIdentifierProvider.Find(null);
      //Invoke Find with null
     }
```

```
    catch(EngineeringTargetInvocationException)
    {
     // new use case and new behavior.
    }
}
```

3.19 Managing exception for missing component in project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

With the TIA Portal Openness v18, You will get a new Siemens.Engineering.MissingProducts exception when you attempting to open a project using any one of the following APIs in a TIA Portal where some of the products or packages are missing.

- Project Open(FileInfo path)
- Project Open(FileInfo path, UmacDelegate umacDelegate)
- Project Open(FileInfo path, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)
- Project OpenWithUpgrade(FileInfo path)
- Project OpenWithUpgrade(FileInfo path, UmacDelegate umacDelegate)
- Project OpenWithUpgrade(FileInfo path, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)
- Project Retrieve(FileInfo sourcePath, DirectoryInfo targetDirectory)
- Project Retrieve(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate)
- Project Retrieve(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)
- Project RetrieveWithUpgrade(FileInfo sourcePath, DirectoryInfo targetDirectory)
- Project RetrieveWithUpgrade(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate)
- Project RetrieveWithUpgrade(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)

Before TIA Portal Openness v18, you can get a Siemens.Engineering.EngineeringTargetInvocation exception.

Program code

```
Copies the following program code to clipboard.
private static void
ManagingExceptionForMissingComponent(ProjectPath
s_ProjectPath)
{
    var projectPath = new FileInfo(s_ProjectPath);
    try
    {
    var project = tiaPortal.Projects.Open(projectPath);
    }
    catch (Siemens.Engineering.MissingProductsException ex)
    {
    Console.WriteLine(ex);
    }
}
```

3.20 Accessing multilingual support

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness application to select and enable their preferred project languages. A new service 'Load to PLC' is added to enable the project language in TIA Portal Openness.

LoadToPLC

Method name	Description
GetAttribute(string attributeName)	To get load to plc.
SetAttribute(string attributeName, bool true)	To enable LoadToPlc for a project language.

Note

Exception for SetAttribute() Action:

- Invalid Configuration: EngineeringTargetInvocationException is thrown for invalid configurations.
- Exceeding Language Limit: ConfigOpennessUserException is thrown if more than 20 languages are set to LoadToPlc.

Program code: Load to PLC

To get load to PLC modify the following program code:

```
Copies the following program code to clipboard.
private void GetLoadToPLC(Project project)
{
    // 1. Example to use 'GetAttribute'
    var project = CreatTestProject();// Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "", "");
    // Create Device
    var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
    var rowOrder = 2;
    var index = rowOrder-1;
    //zeroth index based, index 1 represent 2nd row.
    var tableData = multilingualSupportTable.Rows[index];
    //StructuredDataComposition
   if (tableData != null)
   {
```

```
   var loadToPlc =
(bool)tableData.GetAttribute("LoadToPlc");//StructuredData,
   }
}
```

To set load to PLC modify the following program code:

```
Copies the following program code to clipboard.
private void SetLoadToPLC(Project project)
{
   // 2. Example to use 'SetAttribute'
   var project = CreatTestProject();// Create project
   var plc = PlugDevice(project, plcTypeIdentifier, "",
"");// Create Device
   var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
   var tableData = multilingualSupportTable.Rows[index];
//StructuredDataComposition
   var rowOrder = 2; //Position of language is 2.
   var index = rowOrder-1; // zeroth index based,
   if (tableData != null)
   {
   tableData.SetAttribute("LoadToPlc", true);
;//StructuredData, project language is enabled.
   }
}
```

Language Position

In the new MultilingualSupport, the structured data is based on the project language, making the language position immutable and representing the project language row number.

Note

Exception:

- Setting an invalid language position throws an EngineeringTargetInvocationException.

Active

Use Generic APIs to set or get the Active status of a project language.

- Argument Validation for SetAttribute()
  - **LoadToPlc Dependency**: ConfigOpennessUserException is thrown if Active is set to true while LoadToPlc is false.
  - **Language Limit:** Only 3 languages can be set to Active; exceeding this limit throws ConfigOpennessUserException.



```
Copies the following program code to clipboard.
private void GetActiveStatusProjectLanguage()
{
    // 1. Example to use 'GetAttribute'
    var project = CreatTestProject();
    // Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "", "");
    // Create Device
    var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
    var rowOrder = 2;
    var index = rowOrder-1;
    //zeroth index based, index 1 represent 2nd row.
    var tableData = multilingualSupportTable.Rows[index];
    //StructuredDataComposition
    if (tableData != null)
    {
    var active = (bool)tableData.GetAttribute("Active");
    // get the active status for a project language
```

```
    }
     // 2. Example to use 'SetAttribute'
    var project = CreatTestProject();
    // Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "", "");
    // Create Device
    var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
    var rowOrder = 2;
    //Position of language is 2.
    var index = rowOrder-1;
    // zeroth index based,
    var tableData = multilingualSupportTable.Rows[index];
   //StructuredDataComposition
   if (tableData != null)
   {
   tableData.SetAttribute("LoadToPlc", true);
   //First need to set the loadToPlc as true
   tableData.SetAttribute("Active", true);
   //set the project language active as true
}
```

Display Language

Display languages (e.g., German, English) can be set using the Generic Set/Get API.

- **Exception for SetAttribute()**

  - **Active Dependency**: ConfigOpennessUserException is thrown if a display language is enabled while Active is false

```
Copies the following program code to clipboard.
    // 1. Example to use 'GetAttribute'
    var project = CreatTestProject();// Create project
```

```
    var plc = PlugDevice(project, plcTypeIdentifier, "",
"");// Create Device
    var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
    var rowOrder = 2;
    var index = rowOrder-1; //zeroth index based, index 1
represent 2nd row.
    var tableData = multilingualSupportTable.Rows[index];
//StructuredDataComposition
   if (tableData != null)
   {
    var german= (bool)tableData.GetAttribute("German");
    // get the display language is enabled for a given row or
structured data
   }
   // 2. Example to use 'SetAttribute'
   var project = CreatTestProject();// Create project
   var plc = PlugDevice(project, plcTypeIdentifier, "",
"");// Create Device
   var multilingualSupportTable = plc.
   GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
   var rowOrder = 2; //Position of language is 2.
   var index = rowOrder-1; // zeroth index based,
   var tableData = multilingualSupportTable.Rows[index];
//StructuredDataComposition
   if (tableData != null)
   {
   tableData.SetAttribute("LoadToPlc", true);//set the
loadToPlc as true
   tableData.SetAttribute("Active", true);//set the active as
truth
```

```
   tableData.SetAttribute("German", true);//then only we can
set German as true
   }
```

Creation/Deletion of Structured Data Instance

To add new structured data, a new project language must be added. Directly adding a row to the structured data table is restricted, as these rows are derived from the project languages.